

```
// FIG19_8.CPP
// Driver for class Circle
#include <iostream.h>
#include "point2.h"
#include "circle2.h"

main()
{
    Circle c(2.5, 3.7, 4.3);

    cout << "X coordinate is " << c.getX()
          << "\nY coordinate is " << c.getY()
          << "\nRadius is " << c.getRadius();

    c.setRadius(4.25);
    c.setPoint(2, 2);
    cout << "\n\nThe new location and radius of c are\n"
          << c << "\nArea " << c.area() << endl;

    Point &pRef = c;
    cout << "\nCircle printed as a Point is: "
          << pRef << endl;

    return 0;
}
```

```
X coordinate is 3.7
Y coordinate is 4.3
Radius is 2.5

The new location and radius of c are
Center = [2, 2]; Radius = 4.25
Area: 56.74

Circle printed as a Point is: [2.00, 2.00]
```

Fig. 19.8 Manejador para la clase Circle (parte 3 de 3).

estos son miembros protegidos de la clase base **Circle** (**x** y **y** fueron heredados por **Circle** a partir de **Point**). Note también que es necesario hacer referencia a **x**, **y** y **radius** mediante objetos, como en **c.x**, **c.y** y **c.radius**. Esto es debido a que la función de operador de inserción de flujo homónima no es una función miembro de la clase **Cylinder**, pero es un amigo de la clase. El programa manejador produce un objeto de la clase **Cylinder** y a continuación utiliza las funciones *get* para obtener la información respectiva del objeto **Cylinder**. Otra vez, **main** no es ni una función miembro ni un amigo de la clase **Cylinder**, por lo que no puede hacer referencia directa a los datos protegidos de la clase **Cylinder**. El programa manejador a continuación utiliza funciones *set* de nombre **setHeight**, **setRadius** y **setPoint** para redefinir la altura, el radio y las coordenadas del cilindro. Por último, el manejador hace algo en especial interesante. Inicializa la variable de referencia **pRef** del tipo "referencia al objeto **Point**" (**Point &**) al objeto **cyl** de **Cylinder**. A continuación imprime **pRef** el cual, a pesar

del hecho que ha sido inicializado con un objeto **Cylinder**, "piensa" que se trata de un objeto **Point**, por lo tanto, el objeto **Cylinder** de hecho imprime un objeto **Point**. El manejador a continuación inicializa la variable de referencia **cRef** del tipo "referencia a un objeto **Circle**" (**Circle &**), al objeto **cyl** de **Cylinder**. A continuación imprime **cRef** el cual, a pesar del hecho de que ha sido inicializado con un objeto **Cylinder**, "piensa" que es un objeto **Circle**, por lo que el objeto **Cylinder** de hecho se imprime como un objeto **Circle**. También es extraída el área del **Circle**.

Este ejemplo demuestra muy bien la herencia pública y la definición y referencia de miembros de datos protegidos. A estas alturas el lector deberá sentirse familiarizado con los fundamentos de la herencia. En el siguiente capítulo mostraremos cómo programar con jerarquías de herencia de forma general mediante el polimorfismo. La abstracción de datos, la herencia y el polimorfismo forman el núcleo de la programación orientada a objetos.

19.15 Herencia múltiple

Hasta ahora en el capítulo hemos analizado la herencia simple, en la cual cada clase es derivada de una clase base. Una clase puede ser derivada a partir de más de una clase base; esta derivación se conoce como *herencia múltiple*. La herencia múltiple significa que una clase derivada hereda los miembros de varias clases base. Esta poderosa capacidad fomenta formas interesantes de reutilización de software, pero puede causar una variedad de problemas de ambigüedad.

Práctica sana de programación 19.2

La herencia múltiple, si se utiliza de forma adecuada, es una capacidad poderosa. La herencia múltiple deberá ser utilizada cuando exista una relación "es una" entre un nuevo tipo y dos o más tipos existentes (es decir, tipo A "es un" tipo B y "es un" tipo C).

```
// CYLINDER2.H
// Definition of class Cylinder
#ifndef CYLINDER2_H
#define CYLINDER2_H
#include "circle2.h"

class Cylinder : public Circle {
    friend ostream& operator<<(ostream&, const Cylinder&);
public:
    // default constructor
    Cylinder(float h = 0.0, float r = 0.0,
             float x = 0.0, float y = 0.0);
    void setHeight(float); // set height
    float getHeight() const; // return height
    float area() const; // calculate and return area
    float volume() const; // calculate and return volume
protected:
    float height; // height of the Cylinder
};

#endif
```

Fig. 19.9 Definición de clase Cylinder (parte 1 de 3).

```
// CYLINDER2.CPP
// Member and friend function definitions for class Cylinder
#include <iostream.h>
#include <iomanip.h>
#include "cylindr2.h"

// Cylinder constructor calls Circle constructor
Cylinder::Cylinder(float h, float r, float x, float y)
    : Circle(r, x, y) // call base-class constructor
{ height = h; }

// Set height of Cylinder
void Cylinder::setHeight(float h) { height = h; }

// Get height of Cylinder
float Cylinder::getHeight() const { return height; }

// Calculate area of Cylinder (i.e., surface area)
float Cylinder::area() const
{
    return 2 * Circle::area() +
        2 * 3.14159 * radius * height;
}

// Calculate volume of Cylinder
float Cylinder::volume() const
{ return 3.14159 * radius * radius * height; }

// Output Cylinder dimensions
ostream& operator<<(ostream &output, const Cylinder& c)
{
    output << "Center = [" << c.x << ", " << c.y
        << "]; Radius = " << setiosflags(ios::showpoint)
        << setprecision(2) << c.radius
        << "; Height = " << c.height;

    return output; // enables concatenated calls
}
```

Fig. 19.9 Definiciones de función miembro y de función amigo para la clase **Cylinder** (parte 2 de 3).

Considere el ejemplo de herencia múltiple de la figura 19.10. La clase **Base1** contiene un miembro de datos protegido **int value**. **Base1** contiene un constructor que define **value** y la función miembro pública **getData**, que lee **value**.

La clase **Base2** es similar a la clase **Base1**, excepto que sus datos protegidos son **char letter**. **Base2** también tiene una función miembro pública **getData**, pero su función lee el valor de **char letter**.

La clase **Derived** es heredada tanto de la clase **Base1** como de la clase **Base2** mediante herencia múltiple. **Derived** tiene el miembro de datos privados **float real**, y tiene la función miembro pública **getReal**, que lee el valor de **float real**.

```
// FIG19_9.CPP
// Driver for class Cylinder
#include <iostream.h>
#include <iomanip.h>
#include "point2.h"
#include "circle2.h"
#include "cylindr2.h"

main()
{
    // create Cylinder object
    Cylinder cyl(5.7, 2.5, 1.2, 2.3);

    // use get functions to display the Cylinder
    cout << "X coordinate is " << cyl.getX()
        << "\nY coordinate is " << cyl.getY()
        << "\nRadius is " << cyl.getRadius()
        << "\nHeight is " << cyl.getHeight();

    // use set functions to change the Cylinder's attributes
    cyl.setHeight(10);
    cyl.setRadius(4.25);
    cyl.setPoint(2, 2);
    cout << "\n\nThe new location, radius, "
        << "and height of cyl are:\n" << cyl << endl << "Area: "
        << cyl.area() << " Volume: " << cyl.volume();

    // display the Cylinder as a Point
    Point &pRef = cyl; // pRef "thinks" it is a Point
    cout << "\n\nCylinder printed as a Point is: " << pRef;

    // display the Cylinder as a Circle
    Circle &cRef = cyl; // cRef "thinks" it is a Circle
    cout << "\n\nCylinder printed as a Circle is:\n" << cRef
        << "\nArea: " << cRef.area() << endl;

    return 0;
}
```

```
X coordinate is 1.2
Y coordinate is 2.3
Radius is 2.5
Height is 5.7

The new location, radius, and height of cyl are:
Center = [2, 2]; Radius = 4.25; Height = 10.00
Area: 380.53 Volume: 567.45

Cylinder printed as a Point is: [2.00, 2.00]

Cylinder printed as a Circle is:
Center = [2.00, 2.00]; Radius = 4.25
Area: 56.74
```

Fig. 19.9 Manejador para la clase **Cylinder** (parte 3 de 3).

```
// BASE1.H
// Definition of class Base1
#ifndef BASE1_H
#define BASE1_H

class Base1 {
public:
    Base1(int x) { value = x; }
    int getData() const { return value; }
protected:    // accessible to derived classes
    int value;  // inherited by derived class
};

#endif
```

Fig. 19.10 Definición de la clase **Base1** (parte 1 de 6).

```
// BASE2.H
// Definition of class Base2
#ifndef BASE2_H
#define BASE2_H

class Base2 {
public:
    Base2(char c) { letter = c; }
    char getData() const { return letter; }
protected:    // accessible to derived classes
    char letter; // inherited by derived class
};

#endif
```

Fig. 19.10 Definición de clase **Base2** (parte 2 de 6).

Note qué directo es indicar herencia múltiple haciendo seguir los dos puntos (:) después de **class Derived** con una lista separada por comas de clases base públicas. Note también que para cada una de sus clases base, el constructor **Derived** llama a los constructores de clase base **Base1** y **Base2**, mediante la sintaxis de inicializador de miembro.

El operador de inserción de flujo homónimo para **Derived** utiliza la notación punto sobre el objeto derivado **d** para imprimir **value**, **letter** y **real**. Esta función de operador es un amigo de **Derived**, por lo que **operator<<** puede tener acceso directo al miembro de datos privado **real** de **Derived**. También, dado que este operador es un amigo de una clase derivada, puede tener acceso a los miembros protegidos **value** y **letter** de **Base1** y de **Base2**, respectivamente.

Ahora examinemos el programa manejador en **main**. Creamos el objeto **b1** de clase **Base1** y lo inicializamos al valor **int 10**. Creamos el objeto **b2** de clase **Base2** y lo inicializamos al valor **char 'Z'**. Entonces, creamos el objeto **d** de la clase **Derived** y lo inicializamos para contener el valor **int 7**, el valor **char 'A'** y el valor **float 3.5**.

El contenido de cada uno de estos objetos de clase base es impreso utilizando ligaduras estáticas. Aún cuando existen dos funciones **getData**, no hay ambigüedad en las llamadas, porque hacen referencia directa a la versión **b1** del objeto de **getData** y a la versión **b2** de **getData**.

```
// DERIVED.H
// Definition of class Derived which inherits
// multiple base classes (Base1 and Base2).
#ifndef DERIVED_H
#define DERIVED_H

#include <iostream.h>
#include "base1.h"
#include "base2.h"

// multiple inheritance
class Derived : public Base1, public Base2 {
    friend ostream &operator<<(ostream &, const Derived &);
public:
    Derived(int, char, float);
    float getReal() const;
private:
    float real; // derived class's private data
};

#endif
```

Fig. 19.10 Definición de clase **Derived** (parte 3 de 6).

```
// DERIVED.CPP
// Member function definitions for class Derived
#include "derived.h"

// Constructor for Derived calls constructors for
// class Base1 and class Base2.
Derived::Derived(int i, char c, float f)
    : Base1(i), Base2(c) // call both base-class constructors
{ real = f; }

// Return the value of real
float Derived::getReal() const { return real; }

// Display all the data members of Derived
ostream &operator<<(ostream &output, const Derived &d)
{
    output << "    Integer: " << d.value
        << "\n Character: " << d.letter
        << "\nReal number: " << d.real;

    return output; // enables concatenated calls
}
```

Fig. 19.10 Definición de funciones miembro para la clase **Derived** (parte 4 de 6).

A continuación imprimimos el contenido del objeto **d** de **Derived** mediante ligadura estática. Pero aquí sí existe un problema de ambigüedad, porque este objeto contiene dos funciones **getData**, una heredada de **Base1** y la otra de **Base2**. Este problema es fácil de resolver mediante

```
// FIG19_10.CPP
// Driver for multiple inheritance example
#include <iostream.h>
#include "base1.h"
#include "base2.h"
#include "derived.h"

main()
{
    Base1 b1(10), *base1Ptr;    // create base-class object
    Base2 b2('Z'), *base2Ptr;   // create other base-class object
    Derived d(7, 'A', 3.5);     // create derived-class object

    // print data members of base class objects
    cout << "Object b1 contains integer "
          << b1.getData()
          << "\nObject b2 contains character "
          << b2.getData()
          << "\nObject d contains:\n" << d;

    // print data members of derived class object
    // scope resolution operator resolves getData ambiguity
    cout << "\n\nData members of Derived can be "
          << "accessed individually:\n"
          << "Integer: " << d.Base1::getData()
          << "\n Character: " << d.Base2::getData()
          << "\nReal number: " << d.getReal() << "\n\n";

    cout << "Derived can be treated as an "
          << "object of either base class:\n";

    // treat Derived as a Base1 object
    base1Ptr = &d;
    cout << "base1Ptr->getData() yields "
          << base1Ptr->getData();

    // treat Derived as a Base2 object
    base2Ptr = &d;
    cout << "base2Ptr->getData() yields "
          << base2Ptr->getData() << endl;

    return 0;
}
```

Fig. 19.10 Manejador para el ejemplo de herencia múltiple (parte 5 de 6).

el operador de resolución de alcance binario, como `d.Base1::getData()` para imprimir el `int` en `value`, y `d.Base2::getData()` para imprimir el `char` en `letter`. En `real` el valor `float` es impreso sin ambigüedades mediante la llamada `d.getReal()`.

A continuación demostramos que las relaciones *es una* de herencia simple también son aplicables a la herencia múltiple. Asignamos la dirección del objeto derivado `d` al apuntador de clase base `base1Ptr`, e imprimimos `int value` invocando la función miembro `getData` de `Base1` desde `base1Ptr`. A continuación asignamos la dirección del objeto derivado `d` al

```
Object b1 contains integer 10
Object b2 contains character Z
Object d contains:
    Integer: 7
    Character: A
    Real number: 3.5

Data members of Derived can be accessed individually:
    Integer: 7
    Character: A
    Real number: 3.5

Derived can be treated as an object of either base class:
base1Ptr->getData() yields 7
base2Ptr->getData() yields A
```

Fig. 19.10 Manejador para el ejemplo de herencia múltiple (parte 6 de 6).

apuntador de clase base `base2Ptr` e imprimimos `char letter` invocando la función miembro `getData` de `Base2` partiendo de `base2Ptr`.

Este ejemplo mostró la mecánica de la herencia múltiple e introdujo un problema simple de ambigüedad. La herencia múltiple es un tema complejo, tratado en mayor detalle en textos de C++, como nuestro C++ *How to Program* (Prentice-Hall, 1994).

Resumen

- Una de las claves al poder de la programación orientada a objetos es conseguir la reutilización del software mediante la herencia.
- El programador puede determinar que la nueva clase debe heredar los miembros de datos y las funciones miembro de una clase base previa definida. En este caso, la nueva clase se conoce como la clase derivada.
- Con la herencia simple, una clase se deriva de una sola clase base. En la herencia múltiple, una clase derivada hereda de múltiples clases base (posiblemente no relacionadas).
- Una clase derivada por lo regular añade miembros de datos y funciones miembro propias, por lo que una clase derivada en general tiene una definición mayor que su clase base. Una clase derivada es más específica que su clase base y normalmente representa menos objetos.
- Una clase derivada no puede tener acceso a los miembros privados de su clase base; permitirlo violaría el encapsulado de la clase base. Una clase base puede, sin embargo, tener acceso a los miembros públicos y protegidos de su clase base.
- Al constructor de clase derivada siempre llamará primero al constructor de su clase base, a fin de crear y de inicializar los miembros de la clase base de la clase derivada.

- Los destructores serán llamados en orden inverso a las llamadas de constructor, por lo que un destructor de clase derivada será llamado antes del destructor de su clase base.
- La herencia permite la reutilización del software, lo que ahorra tiempo de desarrollo y fomenta el uso de software de alta calidad previamente probado y depurado.
- La herencia puede ser realizada a partir de bibliotecas de clases existentes.
- En algún momento el software será construido en su mayor parte partiendo de componentes estándar reutilizables, exactamente de la misma forma que como hoy en día se construye la mayor parte del hardware.
- El responsable de una puesta en práctica de una clase derivada no necesita tener acceso al código fuente de una clase base, pero necesita conocer la interfaz de la clase base y ser capaz de enlazar con el código objeto de la clase base.
- Un objeto de una clase derivada puede ser tratado como un objeto de su correspondiente clase base pública. Sin embargo, lo inverso no es cierto.
- Una clase base existe en una relación jerárquica con sus clases derivadas.
- Una clase puede existir por sí misma. Cuando dicha clase es utilizada con el mecanismo de herencia, se convierte ya sea en clase base, que proporciona atributos y comportamientos a otras clases, o la clase se convierte en una clase derivada, que hereda dichos atributos y comportamientos.
- Una jerarquía de herencia puede tener una profundidad arbitraria dentro de las limitaciones físicas de un sistema particular.
- Las jerarquías son herramientas útiles para comprender y administrar la complejidad. Siendo que el software se está haciendo cada vez más complejo, C++ proporciona mecanismos para apoyar estructuras jerárquicas mediante la herencia y el polimorfismo.
- Se puede utilizar una conversión explícita (cast) para convertir un apuntador de clase base a un apuntador de clase derivada. Si dicho apuntador debe ser desreferenciado, primero deberá hacerse que señale a un objeto del tipo de la clase derivada.
- El acceso protegido sirve como un nivel intermedio de protección entre el acceso público y el acceso privado. Se puede tener acceso a los miembros protegidos de una clase base por los miembros y amigos de la clase base y por los miembros y amigos de las clases derivadas; ninguna otra de las funciones pueden tener acceso a los miembros protegidos de la clase base.
- Los miembros protegidos pueden ser utilizados para extender privilegios a clases derivadas, en tanto que niegan dichos privilegios a funciones no de clase y no amigos.
- La herencia múltiple produce jerarquías arborescentes. Estas gráficas no tienen ciclos porque todas las flechas apuntan en la misma dirección. Dichas gráficas se conocen como gráficas acíclicas dirigidas (DAG).
- Al derivar una clase de una clase base, la clase base puede ser declarada **public**, **protected**, o **private**.
- Al derivar una clase de una clase base **public**, los miembros **public** de la clase base se convierten en miembros **public** de la clase derivada, y los miembros **protected** de la clase base se convierten en miembros **protected** de la clase derivada.

- Al derivar una clase de una clase base **protected**, los miembros **public** y **protected** de la clase base se convierten en miembros **protected** de la clase derivada.
- Al derivar una clase de la clase base **private**, los miembros **public** y **protected** de la clase base se convierten en miembros **private** de la clase derivada.
- Una clase base puede ser o una clase base directa de una clase derivada o una clase base indirecta de una clase derivada. Una clase base directa estará de forma explícita listada al declararse la clase derivada. Una clase base indirecta no está listada explícitamente; más bien es herencia a partir de varios niveles hacia arriba en la estructura de jerarquía arbórea.
- Cuando un miembro de clase base es inapropiado para una clase derivada, podemos redefinir dicho miembro en la clase derivada.
- Es importante distinguir entre relaciones “es un” y relaciones “tiene un”. En una relación “tiene un”, un objeto de clase tiene un objeto de otra clase como un miembro. En una relación “es un”, un objeto de una clase de tipo derivado también puede ser tratado como un objeto de tipo de clase base. “Es un” es herencia. “Tiene un” es composición.
- Un objeto de clase derivada puede ser asignado a un objeto de clase base. Este tipo de asignación tiene sentido por que la clase derivada tiene miembros que corresponden a cada uno de los miembros de la clase base. Un apuntador a un objeto de clase derivada puede ser convertido implícitamente en un apuntador a un objeto de clase base.
- Es posible convertir un apuntador de clase base a un apuntador de clase derivada utilizando conversión explícita (cast). El destino debe ser un objeto de clase derivada.
- Una clase base especifica un estado común. Todas las clases derivadas de una clase base heredan las capacidades de dicha clase base. En el proceso de diseño orientado a objetos, el diseñador busca el estado común y lo segrega para formar clases base deseables. Las clases derivadas a continuación son personalizadas más allá de las capacidades heredadas de la clase base.
- Puede ser confuso leer un conjunto de declaraciones de clase derivada, porque no todos los miembros de la clase derivada están presentes en dichas declaraciones. En particular, los miembros heredados no están listados en las declaraciones de clase derivada, pero estos miembros están de hecho presentes en dichas clases derivadas.
- Las relaciones “tiene un” son ejemplos de creación de nuevas clases mediante composición de clase existentes.
- Los constructores de objeto miembro son llamados en el orden en el cual dichos objetos han sido declarados. En la herencia, los constructores de clase base son llamados en el orden en el cual se especificó la herencia, y antes del constructor de la clase derivada.
- Tratándose de un objeto de clase derivada, primero se llama al constructor de clase base, y a continuación el constructor de clase derivada.
- Cuando un objeto de clase derivada sale de alcance, se llaman a los destructores en orden inverso a la de los constructores —primero se llama al destructor de la clase derivada y a continuación al destructor de la clase base.
- Una clase puede ser derivada de más de una clase base; dicha derivación se llama herencia múltiple.

- Indique herencia múltiple haciendo seguir al indicador de herencia de dos puntos (:) con una lista separada por comas de clases base.
- El constructor de clase derivada llama a los constructores de clase base para cada una de sus clases base, mediante la sintaxis de inicializador de miembro.

Terminología

abstracción	herencia
ambigüedad en herencia múltiple	relación <i>es un</i>
clase base	relación <i>conoce a</i>
constructor por omisión de clase base	control de acceso de miembro
constructor de clase base	clase miembro
destructor de clase base	objeto miembro
inicializador de clase base	herencia múltiple
apuntador de clase base	programación orientada a objetos (OOP)
jerarquía de clase	apuntador a un objeto de clase base
biblioteca de clase	apuntador a un objeto de clase derivada
cliente de una clase	clase base privada
composición	herencia privada
personalizar software	clase base protegida
clase derivada	derivación protegida
constructor de clase derivada	palabra reservada protected
destructor de clase derivada	miembro protegido de una clase
apuntador de clase derivada	clase base pública
clase base directa	herencia pública
gráfica acíclica dirigida (DAG)	redefinición de un miembro de clase base
amigos de una clase base	herencia simple
amigos de una clase derivada	reutilización de software
relación <i>tiene un</i>	componentes estándar de software
relación jerárquica	subclase
clase base indirecta	superclase
error de recursión infinito	relación <i>utiliza un</i>

Errores comunes de programación

- 19.1 Puede causar errores tratar un objeto de clase base como si fuera un objeto de clase derivada.
- 19.2 Efectuar la conversión explícita de un apuntador de clase base que señala a un objeto de clase base a un apuntador de clase derivada y a continuación hacer referencia a miembros de clase derivada que no existen en dicho objeto.
- 19.3 Cuando se redefine una función miembro de clase base en una clase derivada, es común hacer que la versión de clase derivada llame a la versión de clase base y haga algún trabajo adicional. Causa recursión infinita no utilizar el operador de resolución de alcance para hacer referencia a la función miembro de la clase base, porque la función miembro de la clase derivada, de hecho, está llamando a sí misma.
- 19.4 Escribir un programa que dependa de que los objetos miembros sean construidos en un orden particular, puede producir errores sutiles.
- 19.5 Asignar un objeto de clase derivada a un objeto de una clase base correspondiente, y a continuación intentar hacer referencia a miembros de sólo la clase derivada en el objeto nuevo de clase base.

- 19.6 Puede causar error convertir en forma explícita (cast) un apuntador de clase base a un apuntador de clase derivada, si a continuación dicho apuntador es utilizado para hacer referencia a un objeto de clase base que no tenga los miembros deseados de la clase derivada.

Prácticas sanas de programación

- 19.1 Al heredar capacidades que no son necesarias en la clase derivada, dichas capacidades se deben enmascarar mediante redefinición de las funciones.
- 19.2 La herencia múltiple, si se utiliza adecuadamente, es una capacidad poderosa. La herencia múltiple deberá ser utilizada cuando exista una relación “es una” entre un nuevo tipo y dos o más tipos existentes (es decir, tipo A “es un” tipo B y “es un” tipo C).

Sugerencia de rendimiento

- 19.1 Si las clases producidas mediante la herencia son más grandes de lo requerido, pudiera dar como resultado un desperdicio de memoria y de recursos de proceso.

Observaciones de ingeniería de software

- 19.1 Una clase derivada no puede tener acceso directo a miembros privados de su clase base.
- 19.2 Una redefinición de una función miembro de clase base en una clase derivada no necesaria debe tener la misma signatura que la función miembro de clase base.
- 19.3 Cuando en una clase derivada se cree un objeto, primero se ejecutará el constructor de clase base, después se ejecutarán los constructores de los objetos miembros correspondientes a la clase derivada, y después se ejecutará el constructor de la clase derivada. Los destructores serán llamados en orden inverso en el cual fueron llamados sus correspondientes constructores.
- 19.4 Crear una clase derivada no afecta el código fuente o el código objeto de su clase base; mediante la herencia se conserva la integridad de la clase base.
- 19.5 En un sistema orientado a objetos, las clases a menudo están íntimamente relacionadas. “Disgregue y elimine” atributos y comportamientos comunes y colóquelos en una clase base. A continuación utilice la herencia para formar clases derivadas.
- 19.6 Una clase derivada contiene los atributos y comportamientos de su clase base. Una clase derivada también puede contener atributos y comportamientos adicionales. Con la herencia, la clase base puede ser compilada independiente de la clase derivada. Para tener la capacidad de combinar estos atributos y comportamientos adicionales con la clase base para formar la clase derivada basta compilar los atributos y comportamientos incrementados de la clase derivada.
- 19.7 Las modificaciones a una clase base no requieren que sea modificada la clase derivada, siempre y cuando la interfaz pública de la clase base se mantenga sin modificación. Pudiera sin embargo, ser necesario recompilar las clases derivadas.
- 19.8 Las modificaciones a una clase miembro no requieren que su clase compuesta que la encierra sea modificada, siempre y cuando se mantenga sin modificación la interfaz pública de la clase miembro. Note que pudiera ser necesario recompilar la clase compuesta.

Ejercicios de autoevaluación

- 19.1 Llene cada uno de los siguientes espacios en blanco:
- a) Si la clase **Alpha** hereda de la clase **Beta**, la clase **Alpha** se llama la clase _____ y la clase **Beta** se llama la clase _____.

- b) C++ proporciona _____, lo que permite que una clase derivada herede de muchas clases base, aún si dichas clases base no están relacionadas.
- c) La herencia permite _____ lo que ahorra tiempo en el desarrollo, y fomenta el uso de software de alta calidad previamente probado.
- d) Un objeto de una clase _____ puede ser tratado como si fuera un objeto de su correspondiente clase _____.
- e) Para convertir un apuntador de clase base a un apuntador de clase derivada deberá utilizarse un _____ porque el compilador considera lo anterior una operación peligrosa.
- f) Los tres especificadores de acceso de miembros son _____, _____ y _____.
- g) Al derivar una clase de una clase base con herencia **public**, los miembros **public** de la clase se convierten en miembros _____ de la clase derivada, y los miembros **protected** de la clase base se convierten en miembros _____ de la clase derivada.
- h) Al derivar una clase de una clase base con herencia **protected**, los miembros **public** de la clase base se convierten en miembros _____ de la clase derivada, y los miembros **protected** de la clase base se convierten en miembros _____ de la clase derivada.
- i) Una relación "tiene un" entre clases representa _____ y una relación "es un" entre clases representa _____.

Respuesta a los ejercicios de autoevaluación

19.1 a) derivado, base. b) herencia múltiple. c) reutilización del software. d) derivada, base. e) conversión explícita (cast). f) **public**, **protected**, **private**. g) **public protected**. h) **protected**, **protected**. i) composición, herencia.

Ejercicios

19.2 Considere la clase **bicycle**. En base en sus conocimientos de algunos componentes básicos y comunes de las bicicletas, muestre una jerarquía de clases en la cual la clase **bicycle** hereda de otras clases, las cuales a su vez también heredan de otras clases. Analice la producción de varios objetos de la clase **bicycle**. Analice la herencia correspondiente a la clase **bicycle** para otras clases derivadas íntimamente relacionadas.

19.3 Defina brevemente cada uno de los términos siguientes: herencia, herencia múltiple, clase base y clase derivada.

19.4 Analice el por qué se considera peligroso para el compilador la conversión del apuntador de clase base a un apuntador de clase derivada.

19.5 Señale todas las formas que le vengan a la mente tanto de dos como de tres dimensiones, y organice estas formas en una jerarquía de formas. Su jerarquía deberá tener una clase base **Shape**, a partir de la cual se derivarán la clase **TwoDimensionalShape** y la clase **ThreeDimensionalShape**. Una vez desarrollada la jerarquía, defina cada una de las clases en la jerarquía. Utilizaremos esta jerarquía en los ejercicios del capítulo 20 para procesar todas las formas como objetos de la clase base **Shape**. Esta es una técnica conocida como polimorfismo.

19.6 Un tipo popular de clase es la que se llama una clase colección o una clase contenedor. Una clase de este tipo contiene elementos de otras clases. Algunos tipos de clases de colección son los arreglos, las pilas, las colas, las listas enlazadas, los árboles, las cadenas de caracteres, los conjuntos, las bolsas, los diccionarios, las tablas, etcétera. Una clase contenedor o de colección proporciona en forma típica servicios como es insertar un elemento, borrarlo y buscarlo, combinar dos conjuntos o colecciones, determinar la intersección de dos colecciones (es decir, los elementos que sean comunes a ambas), imprimir una colección, encontrar el elemento más grande en la colección, encontrar el elemento más pequeño en la colección, encontrar la suma de los elementos de la colección, etcétera.

- a) Haga una lista de todos los tipos de clases de colección que se le puedan ocurrir (incluyendo aquellos que ya hemos mencionado).
- b) Arregle estas clases de colección en una jerarquía de clases. Quizás desee distinguir entre colecciones ordenadas y colecciones desordenadas.
- c) Ponga en práctica todas las clases que pueda, utilizando la herencia para minimizar la cantidad de código nuevo que deba escribir para la creación de cada una de las nuevas clases.
- d) Escriba un programa manejador que pruebe cada una de las clases en su jerarquía de herencia.

20

Funciones virtuales y polimorfismo

Objetivos

- Comprender el concepto de polimorfismo.
- Comprender cómo declarar y utilizar las funciones virtuales para conseguir el polimorfismo.
- Comprender la diferencia entre clases abstractas y clases concretas.
- Aprender a declarar funciones virtuales puras a fin de crear clases abstractas.
- Valorizar cómo el polimorfismo consigue que los sistemas sean extensibles y mantenibles.

¡Oh! tiene usted una repetición maldecible, y es ciertamente capaz de corromper a un santo.

William Shakespeare
Henry IV, Part I, 1, ii

Proposiciones generales no deciden los casos concretos.
Oliver Wendell Holmes

*Un filósofo de imponente estatura no piensa en el vacío.
Inclusive sus más abstractas ideas están, hasta cierto punto,
condicionadas por lo que es o no sabido en la era en que vive.*
Alfred North Whitehead

Sinopsis

- 20.1 Introducción
- 20.2 Campos de tipo y enunciados `switch`
- 20.3 Funciones virtuales
- 20.4 Clases base abstractas y clases concretas
- 20.5 Polimorfismo
- 20.6 Estudio de caso: un sistema de nóminas utilizando polimorfismo
- 20.7 Clases nuevas y ligadura dinámica
- 20.8 Destrucción virtuales
- 20.9 Estudio de caso: cómo heredar interfaz y puesta en práctica

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Observaciones de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

20.1 Introducción

Mediante las *funciones virtuales* y el *polimorfismo*, se hace posible diseñar y poner en práctica sistemas que son de mayor facilidad extensibles. Se pueden escribir programas para procesar objetos de clases existentes y de clases que aun no existan cuando el programa esté en desarrollo. Si esas clases deben ser derivadas de clases base que el programa conozca, éste puede proporcionar un marco general para manipular los objetos de las clases base, y los objetos de las clases derivadas encajarán bien dentro de este marco. De hecho, basándose en este principio, productos completos son elaborados, conocidos como marcos de aplicación.

20.2 Campos de tipo y enunciados `switch`

Una forma de tratar los objetos de muchos tipos distintos es utilizar un enunciado `switch`, a fin de tomar la acción apropiada sobre cada objeto, basándose en su tipo. Al utilizar la lógica `switch` se presentan muchos problemas. Se le podría olvidar al programador llevar a cabo la correspondiente prueba de tipo, cuando sea requerida. También se le podría olvidar probar todos los casos posibles, incluidos en un `switch`. Si se modifica un sistema basado en `switch`, el programador podría olvidar insertar los nuevos casos en los enunciados `switch` existentes. Toda modificación a los enunciados `switch` efectuados para manejar nuevos tipos, exigen que sea modificado cada enunciado `switch` del sistema; rastrear dichos enunciados puede resultar tardado y sujeto a errores.

Como veremos, las funciones virtuales y la programación polimórfica puede eliminar la necesidad de la lógica `switch`. El programador puede utilizar el mecanismo de las funciones virtuales para ejecutar en forma automática la lógica equivalente, evitando por lo tanto los tipos de errores asociados típicamente con la lógica `switch`.

Observación de ingeniería de software 20.1

Una consecuencia interesante del uso de funciones virtuales y polimorfismo es que los programas adquieren una apariencia simplificada. Contienen menos lógica de bifurcación, prefiriendo un código secuencial más sencillo. Esta simplificación facilita probar, depurar y mantener el programa.

20.3 Funciones virtuales

Suponga un conjunto de clases de forma, como **Circle**, **Triangle**, **Rectangle**, **Square**, etcétera, todas ellas derivadas de la clase base **Shape**. En la programación orientada a objetos, cada una de estas clases estaría investida con la capacidad de dibujarse a sí misma. Aunque cada clase tiene su propia función **draw**, la función **draw** correspondiente a cada forma es bien distinta. Cuando se dibuja una forma, cualquiera que ésta sea, sería agradable tener la capacidad de tratar todas estas formas en una genérica, como objetos de la clase base **Shape**. Entonces al dibujar cualquier forma, sólo llamaríamos a la función **draw** de la clase base **Shape**, y dejaríamos que el programa determinase en forma dinámica (es decir, en tiempo de ejecución) cual de las funciones **draw**, de las clases derivadas, utilizar.

Para habilitar este tipo de comportamiento, declaramos **draw** en la clase base en forma de una *función virtual*, a continuación, en cada una de las clases derivadas *redefinimos draw*, a fin de que se dibuje la forma apropiada. Se declara una función virtual en la clase base precediendo el prototipo de función con la palabra reservada **virtual**.

Observación de ingeniería de software 20.2

Una vez declarada una función como virtual, se conserva como virtual a lo largo de toda la jerarquía de herencia, a partir de dicho punto.

Práctica sana de programación 20.1

A pesar que ciertas funciones en forma implícita son virtuales, en razón de una declaración ya hecha en la jerarquía de clase, algunos programadores prefieren declarar estas funciones virtuales en forma explícita en cada uno de los niveles de la jerarquía, a fin de promover claridad en el programa.

Observación de ingeniería de software 20.3

Cuando una clase derivada decide no definir una función virtual, la clase derivada sólo heredará la función virtual de la clase base inmediata.

Si en la clase base la función **draw** ha sido declarada **virtual**, y si entonces utilizamos un apuntador de clase base para señalar al objeto de la clase derivada y utilizando este apuntador invocamos la función **draw**, el programa seleccionará de manera dinámica (es decir, en tiempo de ejecución) la función correcta **draw** de la clase derivada. Esto se conoce como *ligadura dinámica* (vea las figuras 20.1 y 20.2).

Observación de ingeniería de software 20.4

Las funciones virtuales redefinidas deben tener el mismo tipo de regreso y la misma firma que la función virtual base.

Error común de programación 20.1

Resultará un error de sintaxis redefinir una clase derivada como función virtual de clase base, sin asegurarse que la función derivada tenga el mismo tipo de regreso y signatura que la versión de clase base.

Si una función virtual es llamada haciendo referencia a un objeto específico por nombre, y utilizando el operador de selección miembro punto, la referencia se resuelve en tiempo de compilación (esto se llama *ligadura estática*) y la función virtual llamada es aquella definida para, (o heredada por) la clase de dicho objeto en particular.

La homonimia no utiliza ligadura dinámica. Más bien, la homonimia es resuelta en tiempo de compilación al seleccionar la definición de función con una firma que coincida con la llamada de función (utilizando posiblemente una conversión implícita cast de tipo para que la coincidencia ocurra). Esto corresponde a ligadura estática.

20.4 Clases base abstractas y clases concretas

Cuando pensamos sobre una clase como un tipo, suponemos que serán producidos objetos de dicho tipo. Existen, sin embargo, muchas situaciones en las cuales resulta útil definir clases para las cuales el programador no tiene intención de producir ningún objeto. Estas clases se conocen como *clases abstractas*. Dado que en situaciones de herencia son utilizadas como clases base, a menudo nos referiremos a ellas como *clases base abstractas*. A partir de una clase base abstracta no se pueden producir objetos.

El único fin de una clase abstracta es proporcionar una clase base apropiada, a partir de la cual las clases pueden heredar interfaz y/o puesta en práctica. Las clases a partir de las cuales los objetos se pueden producir, se conocen como *clases concretas*.

Podríamos tener una clase base abstracta `TwoDimensionalObject` y derivar clases concretas, como `Square`, `Circle`, `Triangle`, etcétera. También podríamos tener una clase base abstracta `ThreeDimensionalObject` y derivar clases concretas como `Cube`, `Sphere`, `Cylinder`, `Pyramid`, etcétera. Estas clases base abstractas resultan demasiado genéricas para definir objetos reales; antes de pensar en la producción de objetos necesitamos ser más específicos. Y esto es lo que hacen las clases concretas; dan la definición que convierte en razonable la producción de objetos.

Una clase con funciones virtuales se hace abstracta al declarar uno o más de sus funciones virtuales como puras. Una *función virtual pura* es una que en su declaración contenga un inicializador de `= 0`, como en

```
virtual float earnings() const = 0;    // pure virtual
```

Observación de ingeniería de software 20.5

Si una clase se deriva de una clase con una función virtual pura, y para dicha función virtual pura no se ha dado definición en la clase derivada, entonces la función virtual también se conserva pura en la clase derivada. Por consecuencia, también la clase derivada será una clase abstracta.

Error común de programación 20.2

Es un error de sintaxis intentar producir un objeto a partir de una clase abstracta (es decir, de una clase que contenga una o más funciones virtuales puras).

Una jerarquía no necesita contener ninguna clase abstracta, pero como veremos, muchos sistemas de calidad orientados a objetos tienen jerarquías de clase encabezadas por una clase abstracta. En algunos casos, las clases abstractas forman los primeros niveles superiores de la jerarquía. Un buen ejemplo de lo anterior es la jerarquía de forma. La jerarquía podría ser encabezada por la clase base abstracta `Shape`. En el siguiente nivel inferior, podríamos tener dos

clases base más, de tipo abstracto, es decir `TwoDimensionalShape` y `ThreeDimensionalShape`. El siguiente nivel hacia abajo empezaría a definir clases concretas correspondientes a formas de dos dimensiones, como círculos, cuadrados y clases concretas correspondientes a formas tridimensionales, como esferas y cubos.

20.5 Polimorfismo

C++ permite el *polimorfismo* —la capacidad de objetos de clases diferentes, relacionados mediante la herencia, a responder de forma distinta a una misma llamada de función miembro. Si, por ejemplo, la clase `Rectangle` es derivada de la clase `Quadrilateral`, entonces un objeto `Rectangle` es una versión más específica de un objeto `Quadrilateral`. Una operación (como el cálculo del perímetro o de la superficie) que pueda ser efectuada en un objeto de la clase `Quadrilateral`, también podrá ser ejecutada en un objeto de la clase `Rectangle`.

El polimorfismo se pone en práctica vía las funciones virtuales. Cuando se hace una solicitud, a través de un apuntador de clase base (o una referencia) para usar una función virtual, C++ escoge la función redefinida correcta, en la clase derivada apropiada, asociada con el objeto.

Algunas veces, una función miembro no virtual es definida en una clase base y redefinida en una derivada. Si una función miembro como ésta es llamada mediante un apuntador de clase base, se utilizará la versión de clase base. Si la función miembro es llamada mediante un apuntador de clase derivada, se utilizará la versión de clase derivada. Esto es comportamiento no polimórfico.

Mediante el uso de las funciones virtuales y del polimorfismo, una llamada de función miembro puede hacer que ocurran distintas acciones, dependiendo del tipo del objeto que reciba la llamada. Esto le da al programador una capacidad de expresión tremenda. En las siguientes secciones veremos ejemplos del poder del polimorfismo y de las funciones virtuales.

Observación de ingeniería de software 20.6

Mediante las funciones virtuales y el polimorfismo, el programador puede ocuparse de generalidades y dejar que en tiempo de ejecución, el entorno se preocupe de lo específico. El programador puede dirigir una amplia variedad de objetos haciendo que se comporten de formas apropiadas a dichos objetos incluso sin conocer los tipos de los mismos.

Observación de ingeniería de software 20.7

El polimorfismo fomenta la extensibilidad: software escrito para invocar comportamiento polimórfico se escribe en forma independiente del tipo de los objetos a los cuales los mensajes son enviados. Por lo tanto, nuevos tipos de objetos, que pudieran responder a mensajes existentes, pueden ser añadidos en dicho sistema sin modificar el sistema base. A excepción de la parte de código cliente que produce nuevos objetos, los programas no necesitan ser recompilados.

Observación de ingeniería de software 20.8

Una clase abstracta define una interfaz para los distintos miembros de una jerarquía de clase. La clase abstracta contiene funciones virtuales puras, que serán definidas en las clases derivadas. Mediante el polimorfismo todas las funciones en la jerarquía pueden utilizar esta misma interfaz.

A pesar de que no podemos producir objetos de clases base abstractas, sí podemos declarar apuntadores a clases base abstractas. Dichos apuntadores podrán ser utilizados después para permitir manipulaciones polimórficas de objetos de clases derivadas, cuando dichos objetos son producidos a partir de clases concretas.

Consideremos aplicaciones del polimorfismo y de las funciones virtuales. Un administrador de pantalla necesita mostrar una variedad de objetos, incluyendo nuevos tipos de objetos, que serán añadidos al sistema, aún después de que haya sido escrito el administrador de pantalla. El sistema pudiera necesitar mostrar varias formas (es decir, la clase base es **Shape**) como cuadrados, círculos, triángulos, rectángulos y similar (cada una de estas clases de formas es derivada de la clase base **Shape**). El administrador de pantalla utiliza apuntadores de clase base para administrar todos los objetos a mostrar. Para dibujar cualquier objeto (independiente del nivel en la jerarquía de herencia en el cual aparezca dicho objeto), el administrador de pantalla utiliza un apuntador de clase base a dicho objeto, y envía al objeto un mensaje **draw**. En la clase base **Shape** la función **draw** ha sido declarada como virtual pura y en cada una de las clases derivadas ha sido redefinida. Cada objeto sabe como dibujarse a sí mismo. El administrador de pantalla no tiene que preocuparse sobre estos detalles; le indica a cada objeto que se dibuje a sí mismo.

El polimorfismo es en particular efectivo para la puesta en práctica de sistemas de software en capas. Por ejemplo, en los sistemas operativos, cada tipo de dispositivo físico pudiera operar en forma bastante distinta a los demás. Independiente de lo anterior, órdenes o comandos para leer o escribir datos de y hacia los dispositivos pueden tener una cierta uniformidad. El mensaje **write** enviado a un objeto manejador de dispositivo, necesita ser interpretado en específico en el contexto de dicho manejador de dispositivo, y de la forma en que dicho manejador de dispositivo manipula dispositivos de un tipo específico. Pero la llamada **write** por sí misma en realidad no es distinta de la llamada **write** para cualquier otro dispositivo —simplemente colocar cierto número de bytes de la memoria en dicho dispositivo. Un sistema operativo orientado a objetos pudiera utilizar una clase base abstracta para proporcionar una interfaz apropiada para todos los manejadores de dispositivo. Entonces, mediante herencia, a partir de dicha clase base abstracta, se formarían clases derivadas, todas operando en forma similar. Las capacidades (es decir, la interfaz pública) ofrecida por los manejadores de dispositivo se dan en la clase base abstracta como funciones virtuales puras. Las puestas en práctica de estas funciones virtuales se dan en las clases derivadas, correspondientes a los tipos específicos de manejadores de dispositivo.

En el capítulo 17, introdujimos el concepto de los iteradores. Es común definir una *clase iterador*, que recorra todos los objetos de una colección. Si por ejemplo, usted desea imprimir una lista de objetos en una lista enlazada, se puede producir un objeto iterador, que, cada vez que se llame al iterador, devolverá el siguiente elemento de la lista enlazada. Los iteradores son por lo general usados en la programación polimórfica para recorrer una lista enlazada de objetos, correspondiente a varios niveles de una jerarquía. Los apuntadores a una lista como ésta, serían todos apuntadores de clase base.

20.6 Estudio de caso: un sistema de nóminas utilizando polimorfismo

Usemos funciones virtuales y polimorfismo para llevar a cabo cálculos de nómina, basados en el tipo de un empleado (figura 20.1). Utilizamos una clase base **Employee**. Las clases derivadas de **Employee** son **Boss**, mismo que se le paga un salario semanal fijo, independiente del número de horas trabajadas, **CommissionWorker**, que obtiene un salario base fijo más un porcentaje de las ventas, **PieceworkWorker**, a quien se le paga según el número de elementos producidos, y **HourlyWorker**, que cobra por hora y recibe paga por tiempo extraordinario.

Una llamada de función **earnings** es ciertamente aplicable en forma genérica a todos los empleados. Pero la forma en que se calculen los ingresos de cada persona dependerá de la clase del empleado y estas clases son todas derivadas de la clase base **Employee**. Por lo tanto, en la clase base **Employee**, **earnings** es declarado **virtual**, y se dan puestas en práctica apropiadas

de **earnings** para cada una de las clases derivadas. A continuación, a fin de calcular los ingresos de cada empleado, el programa sólo utiliza un apuntador de clase base al objeto de dicho empleado, e invoca la función **earnings**. En un sistema real de nóminas, los distintos objetos “empleado” pudieran estar almacenados en una lista enlazada, en la cual, cada uno de los apuntadores es del tipo **Employee ***. El programa entonces sólo recorrería la lista enlazada, nodo por nodo, utilizando los apuntadores **Employee *** a fin de invocar la función **earnings** sobre cada objeto.

Veamos la clase **Employee** (figura 20.1, parte 1 y 2). Las funciones miembro públicas incluyen un constructor, que toma como argumentos el nombre y el apellido; un destructor, que recupera la memoria asignada dinámicamente; una función **get**, que devuelve el nombre; una función **get**, que devuelve el apellido; y, por último, dos funciones virtuales puras **earnings** y **print**. ¿Por qué estas funciones son virtuales puras? La respuesta es que no tiene sentido hacer puestas en práctica de dichas funciones en la clase **Employee**. Al hacer estas funciones virtuales puras, estamos indicando que proporcionaremos puestas en práctica en las clase derivada, pero no en la clase base misma. No es posible calcular los ingresos de un empleado genérico —primero tenemos que saber qué tipo de empleado es— ni tampoco, a partir de un empleado genérico, podemos imprimir el tipo de empleado.

La clase **Boss** (figura 20.1, parte 3 y 4) es derivada de **Employee** mediante herencia pública. Las funciones miembro públicas incluyen un constructor, que toma como argumentos un nombre, un apellido y un salario semanal, y pasa el nombre y el apellido al constructor **Employee** para inicializar los miembros **firstName** y **lastName** de la parte de clase base del objeto de clase derivada; una función **set**, para asignar un valor nuevo al miembro de datos privado **weeklySalary**; una función virtual **earnings**, que define cómo calcular los ingresos **Boss**; y una función virtual **print**, que extrae el tipo del empleado, seguido por el nombre del mismo.

```
// EMPLOY2.H
// Abstract base class Employee
#ifndef EMPLOY2_H
#define EMPLOY2_H

class Employee {
public:
    Employee(const char *, const char *);
    ~Employee();
    const char *getFirstName() const;
    const char *getLastName() const;

    // Pure virtual functions make
    // Employee an abstract base class.
    virtual float earnings() const = 0; // pure virtual
    virtual void print() const = 0;    // pure virtual
private:
    char *firstName;
    char *lastName;
};

#endif
```

Fig. 20.1 Clase base abstracta **Employee** (parte 1 de 12).

```

// EMPLOY2.CPP
// Member function definitions for
// abstract base class Employee.
//
// Note: No definitions given for pure virtual functions.
#include <iostream.h>
#include <string.h>
#include <assert.h>
#include "employ2.h"

// Constructor dynamically allocates space for the
// first and last name and uses strcpy to copy
// the first and last names into the object.
Employee::Employee(const char *first, const char *last)
{
    firstName = new char[ strlen(first) + 1 ];
    assert(firstName != 0);    // test that new worked
    strcpy(firstName, first);

    lastName = new char[ strlen(last) + 1 ];
    assert(lastName != 0);    // test that new worked
    strcpy(lastName, last);
}

// Destructor deallocates dynamically allocated memory
Employee::~Employee()
{
    delete [] firstName;
    delete [] lastName;
}

// Return a pointer to the first name
const char *Employee::getFirstName() const
{
    // Const prevents caller from modifying private data.
    // Caller should copy returned string before destructor
    // deletes dynamic storage to prevent undefined pointer.

    return firstName;    // caller must delete memory
}

// Return a pointer to the last name
const char *Employee::getLastName() const
{
    // Const prevents caller from modifying private data.
    // Caller should copy returned string before destructor
    // deletes dynamic storage to prevent undefined pointer.

    return lastName;    // caller must delete memory
}

```

Fig. 20.1 Definiciones de función miembro para la clase base abstracta **Employee** (parte 2 de 12).

```

// BOSS1.H
// Boss class derived from Employee
#ifndef BOSS1_H
#define BOSS1_H
#include "employ2.h"

class Boss : public Employee {
public:
    Boss(const char *, const char *, float = 0.0);
    void setWeeklySalary(float);
    virtual float earnings() const;
    virtual void print() const;
private:
    float weeklySalary;
};

#endif

```

Fig. 20.1 Clase **Boss** derivada de la clase base abstracta **Employee** (parte 3 de 12).

```

// BOSS1.CPP
// Member function definitions for class Boss
#include <iostream.h>
#include "boss1.h"

// Constructor function for class Boss
Boss::Boss(const char *first, const char *last, float s)
    : Employee(first, last) // call base-class constructor
{ weeklySalary = s > 0 ? s : 0; }

// Set the Boss's salary
void Boss::setWeeklySalary(float s)
{ weeklySalary = s > 0 ? s : 0; }

// Get the Boss's pay
float Boss::earnings() const { return weeklySalary; }

// Print the Boss's name
void Boss::print() const
{
    cout << "\n          Boss: " << getFirstName()
          << " " << getLastName();
}

```

Fig. 20.1 Definiciones de función miembro para la clase **Boss** (parte 4 de 12).

La clase **CommissionWorker** (figura 20.1, parte 5 de 6) se deriva de **Employee** mediante herencia pública. Las funciones miembro públicas incluyen un constructor, que toma como argumentos un nombre, un apellido, un salario, una comisión y una cantidad de elementos vendidos y pasa el nombre y el apellido al constructor **Employee**, a fin de inicializar los miembros **firstName** y **lastName** de la parte de clase base del objeto de clase derivada; funciones **set**, para asignar nuevos valores a los miembros de datos privados **salary**, **commission** y **quantity**;

```

// COMMIS1.H
// CommissionWorker class derived from Employee
#ifndef COMMIS1_H
#define COMMIS1_H
#include "employ2.h"

class CommissionWorker : public Employee {
public:
    CommissionWorker(const char *, const char *,
                     float = 0.0, float = 0.0, int = 0);
    void setSalary(float);
    void setCommission(float);
    void setQuantity(int);
    virtual float earnings() const;
    virtual void print() const;
private:
    float salary;      // base salary per week
    float commission;  // amount per item sold
    int quantity;      // total items sold for week
};

#endif

```

Fig. 20.1 Clase **CommissionWorker** derivada de la clase base abstracta **Employee** (parte 5 de 12).

una función virtual **earnings**, que define cómo calcular los ingresos **CommissionWorker**; y una función virtual **print**, que extrae el tipo del empleado seguido por el nombre del mismo.

La clase **PieceWorker** (figura 20.1, partes 7 y 8) se deriva de **Employee** mediante herencia pública. Las funciones miembro públicas incluyen un constructor, que toma como argumentos un nombre, un apellido, un salario por pieza y una cantidad de elementos producidos, y pasa el nombre y apellido al constructor **Employee**, a fin de inicializar los miembros **firstName** y **lastName** de la parte de clase base del objeto de clase derivada; funciones **set**, para asignar nuevos valores a los miembros de datos privados **wagePerPiece** y **quantity**; una función virtual **earnings**, que define cómo calcular los ingresos **PieceWorker**; y una función virtual **print**, que extrae el tipo del empleado seguido por el nombre del mismo.

La clase **HourlyWorker** (figura 20.1, partes 9 y 10) se deriva de **Employee** mediante herencia pública. Las funciones miembro públicas incluyen un constructor, que toma como argumentos un nombre, apellido, un salario, el número de horas trabajadas y pasa el nombre y el apellido al constructor **Employee**, a fin de inicializar los miembros **firstName** y **lastName** de la parte de clase base del objeto de clase derivada; funciones **set**, a fin de asignar nuevos valores a los miembros de datos privados **wage** y **hours**; una función virtual **earnings**, que define cómo calcular los ingresos **HourlyWorker**; y una función virtual **print**, que extrae el tipo del empleado seguido por el nombre del mismo.

El programa manejador (figura 20.1 partes 11 y 12) empieza declarando el apuntador de clase base, **ptr**, del tipo **Employee ***. Cada uno de los tres segmentos de código en **main** es similar, por lo que analizaremos sólo el primer segmento, que trata del objeto **Boss**. La línea

```
Boss b("John", "Smith", 800.00);
```

```

// COMMIS1.CPP
// Member function definitions for class CommissionWorker
#include <iostream.h>
#include "commis1.h"

// Constructor for class CommissionWorker
CommissionWorker::CommissionWorker(const char *first,
                                   const char *last, float s, float c, int q)
    : Employee(first, last) // call base-class constructor
{
    salary = s > 0 ? s : 0;
    commission = c > 0 ? c : 0;
    quantity = q > 0 ? q : 0;
}

// Set CommissionWorker's weekly base salary
void CommissionWorker::setSalary(float s)
{ salary = s > 0 ? s : 0; }

// Set CommissionWorker's commission
void CommissionWorker::setCommission(float c)
{ commission = c > 0 ? c : 0; }

// Set CommissionWorker's quantity sold
void CommissionWorker::setQuantity(int q)
{ quantity = q > 0 ? q : 0; }

// Determine CommissionWorker's earnings
float CommissionWorker::earnings() const
{ return salary + commission * quantity; }

// Print the CommissionWorker's name
void CommissionWorker::print() const
{
    cout << "\nCommission worker: " << getFirstName()
          << ' ' << getLastName();
}

```

Fig. 20.1 Definiciones de función miembro para la clase **CommissionWorker** (parte 6 de 12).

produce el objeto de clase derivada **b** de la clase **Boss** y proporciona varios argumentos de constructor incluyendo un nombre, un apellido y un salario fijo semanal. La línea

```
ptr = &b; // base-class pointer to derived-class object
```

coloca la dirección de la clase derivada del objeto **b** en el apuntador de clase base **ptr**. Esto es precisamente lo que debemos hacer para utilizar comportamiento polimórfico. La línea

```
ptr->print(); // dynamic binding
```

invoca a la función miembro **print** del objeto al cual señala **ptr**. Dado que **print** ha sido declarado como una función virtual de la clase base, el sistema invoca la función **print** del

```
// PIECE1.H
// PieceWorker class derived from Employee
#ifndef PIECE1_H
#define PIECE1_H
#include "employ2.h"

class PieceWorker : public Employee {
public:
    PieceWorker(const char *, const char *,
                float = 0.0, int = 0);
    void setWage(float);
    void setQuantity(int);
    virtual float earnings() const;
    virtual void print() const;

private:
    float wagePerPiece; // wage for each piece output
    int quantity;       // output for week
};

#endif
```

Fig. 20.1 Clase **PieceWorker** derivada de la clase base abstracta **Employee** (parte 7 de 12).

objeto de clase derivada —otra vez precisamente lo que llamamos comportamiento polimórfico. Esta llamada de función es un ejemplo de ligadura dinámica— la función es invocada a través de un apuntador de clase base, por lo que la decisión de cuál función deberá invocarse se pospone hasta el tiempo de ejecución. La línea

```
cout << " earned $" << ptr->earnings(); // dynamic binding
```

invoca la función miembro **earnings** del objeto al cual señala **ptr**. Dado de que **earnings** ha sido declarada una función virtual de la clase base, el sistema invoca a la función **earnings** del objeto de clase derivada. También éste es un ejemplo de ligadura dinámica. La línea

```
b.print(); // static binding
```

invoca en forma explícita a la función **print** de la función miembro **print**, mediante el uso del operador de selección miembro punto sobre el objeto **b** específico de **Boss**. Esto es un ejemplo de ligadura estática, porque en tiempo de compilación el tipo del objeto para el cual se llama a la función es ya conocido. Esta llamada ha sido incluida para efectos de comparación con el fin de ilustrar que la función **print** correcta es invocada mediante el uso de la ligadura dinámica. La línea

```
cout << " earned $" << b.earnings(); // static binding
```

invoca de manera explícita la versión **Boss** de la función **earnings**, mediante el uso del operador de selección de miembro punto sobre el objeto **b** específico de **Boss**. También este es un ejemplo de ligadura estática. También se incluye esta llamada para efectos de comparación, a fin de ilustrar que la función **earnings** correcta es invocada utilizando ligadura dinámica.

```
// PIECE1.CPP
// Member function definitions for class PieceWorker

#include <iostream.h>
#include "piece1.h"

// Constructor for class PieceWorker
PieceWorker::PieceWorker(const char *first, const char *last,
                        float w, int q)
    : Employee(first, last) // call base-class constructor
{
    wagePerPiece = w > 0 ? w : 0;
    quantity = q > 0 ? q : 0;
}

// Set the wage
void PieceWorker::setWage(float w)
{ wagePerPiece = w > 0 ? w : 0; }

// Set the number of items output
void PieceWorker::setQuantity(int q)
{ quantity = q > 0 ? q : 0; }

// Determine the PieceWorker's earnings
float PieceWorker::earnings() const
{ return quantity * wagePerPiece; }

// Print the PieceWorker's name
void PieceWorker::print() const
{
    cout << "\n    Piece worker: " << getFirstName()
        << " " << getLastName();
}
```

Fig. 20.1 Definiciones de función miembro para la clase **PieceWorker** (parte 8 de 12).

20.7 Clases nuevas y ligadura dinámica

El polimorfismo y las funciones virtuales ciertamente funcionarían bien en un mundo en el cual todas las clases posibles fueran conocidas con antelación. Pero también funcionan cuando en forma regular a los sistemas se les añaden nuevos tipos de clases.

Se les hace sitio a las nuevas clases mediante la ligadura dinámica (también conocido como *ligadura tardía*). Para que sea compilada una llamada de función virtual, no es necesario conocer el tipo de un objeto en tiempo de compilación. La llamada de función virtual se hace coincidir en tiempo de ejecución con la función miembro del objeto llamado.

Un programa administrador de pantalla puede ahora manejar nuevos objetos de exhibición, conforme sean añadidos al sistema, sin necesidad de recompilación. La llamada de función **draw** se conserva idéntica. Los nuevos objetos mismos contienen sus propias capacidades de dibujo. Esto facilita añadir con un impacto mínimo nuevas capacidades a sistemas. También promueve la reutilización del software.

```
// HOURLY1.H
// Definition of class HourlyWorker
#ifndef HOURLY1_H
#define HOURLY1_H
#include "employ2.h"

class HourlyWorker : public Employee {
public:
    HourlyWorker(const char *, const char *,
                 float = 0.0, float = 0.0);
    void setWage(float);
    void setHours(float);
    virtual float earnings() const;
    virtual void print() const;
private:
    float wage;    // wage per hour
    float hours;   // hours worked for week
};

#endif
```

Fig. 20.1 La clase **HourlyWorker** derivada de la clase base abstracta **Employee** (parte 9 de 12).

```
// HOURLY1.CPP
// Member function definitions for class HourlyWorker
#include <iostream.h>
#include "hourly1.h"

// Constructor for class HourlyWorker
HourlyWorker::HourlyWorker(const char *first, const char *last,
                           float w, float h)
    : Employee(first, last)    // call base-class constructor
{
    wage = w > 0 ? w : 0;
    hours = h >= 0 && h < 168 ? h : 0;
}

// Set the wage
void HourlyWorker::setWage(float w) { wage = w > 0 ? w : 0; }

// Set the hours worked
void HourlyWorker::setHours(float h)
{ hours = h >= 0 && h < 168 ? h : 0; }

// Get the HourlyWorker's pay
float HourlyWorker::earnings() const { return wage * hours; }

// Print the HourlyWorker's name
void HourlyWorker::print() const
{
    cout << "\n    Hourly worker: " << getFirstName()
          << " " << getLastName();
}
```

Fig. 20.1 Definiciones de función miembro para las clases **HourlyWorker** (parte 10 de 12).

```
// FIG20_1.CPP
// Driver for Employee hierarchy

#include <iostream.h>
#include <iomanip.h>
#include "employ2.h"
#include "boss1.h"
#include "commis1.h"
#include "piece1.h"
#include "hourly1.h"

main()
{
    // set output formatting
    cout << setiosflags(ios::showpoint) << setprecision(2);

    Employee *ptr;    // base-class pointer

    Boss b("John", "Smith", 800.00);
    ptr = &b;    // base-class pointer to derived-class object
    ptr->print();    // dynamic binding
    cout << " earned $" << ptr->earnings();    // dynamic binding
    b.print();    // static binding
    cout << " earned $" << b.earnings();    // static binding

    CommissionWorker c("Sue", "Jones", 200.0, 3.0, 150);
    ptr = &c;    // base-class pointer to derived-class object
    ptr->print();    // dynamic binding
    cout << " earned $" << ptr->earnings();    // dynamic binding
    c.print();    // static binding
    cout << " earned $" << c.earnings();    // static binding

    PieceWorker p("Bob", "Lewis", 2.5, 200);
    ptr = &p;    // base-class pointer to derived-class object
    ptr->print();    // dynamic binding
    cout << " earned $" << ptr->earnings();    // dynamic binding
    p.print();    // static binding
    cout << " earned $" << p.earnings();    // static binding

    HourlyWorker h("Karen", "Price", 13.75, 40);
    ptr = &h;    // base-class pointer to derived-class object
    ptr->print();    // dynamic binding
    cout << " earned $" << ptr->earnings();    // dynamic binding
    h.print();    // static binding
    cout << " earned $" << h.earnings();    // static binding

    cout << endl;

    return 0;
}
```

Fig. 20.1 Jerarquía de derivación de clase "empleado" que utiliza una clase base abstracta (parte 11 de 12).

```

Boss: John Smith earned $800.00
Boss: John Smith earned $800.00
Commission worker: Sue Jones earned $650.00
Commission worker: Sue Jones earned $650.00
Piece worker: Bob Lewis earned $500.00
Piece worker: Bob Lewis earned $500.00
Hourly worker: Karen Price earned $550.00
Hourly worker: Karen Price earned $550.00

```

Fig. 20.1 Jerarquía de derivación de clase "empleado" que utiliza una clase base abstracta (parte 12 de 12).

La ligadura dinámica permite que fabricantes independientes de software (ISV), distribuyan software sin tener que revelar secretos propietarios. Estas distribuciones de software pueden estar formadas de solo los archivos de cabecera y archivos objeto. No es necesario revelar el código fuente. Los desarrolladores de software entonces pueden utilizar la herencia, para derivar nuevas clases a partir de aquellas proporcionadas por los ISV. El software, que funciona con las clases que proporcionan los ISV, continuará funcionando con las clases derivadas y utilizará (vía ligaduras dinámicas) las funciones virtuales redefinidas proporcionadas en dichas clases.

La ligadura dinámica requiere que en tiempo de ejecución, se encamine la llamada a una función miembro virtual a la versión de función virtual apropiada para la clase. Se pone en práctica una *tabla de funciones virtuales* llamada la *vtable*, en forma de un arreglo que contiene apuntadores de función. Cada clase que contenga funciones virtuales tiene una *vtable*. Para cada función virtual dentro de la clase, la *vtable* tiene una entrada que contiene un apuntador de función a la versión de la función virtual para uso de un objeto de dicha clase. La función virtual a usar para una clase particular podría ser la función definida en dicha clase, o podría ser una función heredada, ya sea directa o indirecta, de una clase base más alta dentro de la jerarquía.

Cuando una clase base proporciona una función miembro y la declara **virtual**, las clases derivadas pueden redefinir la función virtual, pero no están obligadas a redefinirla. Por lo tanto, una clase derivada puede utilizar la versión de clase base de una función miembro virtual, y esto quedaría indicado en la *vtable*.

Cada objeto de una clase con funciones virtuales contiene un apuntador a la *vtable* para dicha clase. Este apuntador no es accesible para el programador. Se obtiene el apuntador apropiado de función en la *vtable* y se desreferencia para completar la llamada en tiempo de ejecución.

Esta búsqueda y desreferenciación de apuntador en *vtable* requiere de una sobrecarga nominal en tiempo de ejecución.

Sugerencia de rendimiento 20.1

El polimorfismo, tal y como se pone en práctica mediante funciones virtuales y ligaduras dinámicas, resulta eficiente. Los programadores pueden utilizar estas capacidades con un impacto solo nominal sobre el rendimiento del sistema.

Sugerencia de rendimiento 20.2

Las funciones virtuales y las ligaduras dinámicas permiten la programación polimórfica, en contraste con la programación lógica *switch*. Los compiladores optimizantes de C++ por lo regular generan código que se ejecuta con una eficiencia por lo menos igual a la de la lógica basada en *switch* codificada manualmente.

20.8 destructores virtuales

Puede ocurrir un problema, al utilizar el polimorfismo para procesar objetos dinámicamente asignados de una jerarquía de clase. Si es destruido, aplicando el operador **delete** a un apuntador de clase base al objeto, la función destructor de clase base será llamada sobre el objeto. Esto ocurrirá independiente del tipo del objeto al cual está señalado el apuntador de clase base e independiente del hecho que el destructor de cada clase tenga un nombre distinto.

Existe una solución simple para este problema —declarar virtual el destructor de clase base. Esto hará virtuales automáticamente a todos los destructores de clase derivada, a pesar de que no tengan el mismo nombre que el destructor de clase base. Ahora, si un objeto en la jerarquía es destruido en forma explícita, mediante la aplicación del operador **delete** a un apuntador de clase base a un objeto de clase derivada, se llamará al destructor de la clase apropiada.

Práctica sana de programación 20.2

Si una clase tiene funciones virtuales, incluya un destructor virtual, inclusive si para dicha clase no se requiere uno. Las clases que se deriven de esta clase pudieran contener destructores que deberán ser llamados en forma correcta.

Error común de programación 20.3

Declarar un constructor como función virtual. Los constructores no pueden ser virtuales.

20.9 Estudio de caso: cómo heredar interfaz, y puesta en práctica

Nuestro siguiente ejemplo (figura 20.2) vuelve a examinar la jerarquía de punto, círculo y cilindro del capítulo anterior, excepto que ahora encabezamos la jerarquía con la clase base abstracta del capítulo anterior, excepto que ahora encabezamos la jerarquía con la clase base abstracta **Shape**. **Shape** tiene una función virtual pura —**printShapeName**— de tal forma que **Shape** es una clase base abstracta. **Shape** contiene otras dos funciones virtuales, es decir **area** y **volume**, cada una de las cuales tiene una puesta en práctica que regresa un valor cero. **Point** hereda estas puestas en práctica de **Shape**. Esto tiene sentido, porque tanto el área como el volumen de un punto son cero. **Circle** hereda la función **volume** de **Point**, pero **Circle** proporciona su propia puesta en práctica correspondiente a la función **area**. **Cylinder** proporciona sus propias puestas en práctica, tanto para la función **area** como para la función **volume**.

Note que aunque **Shape** es una clase base abstracta, aún así contiene puestas en práctica de dichas funciones miembro, y dichas puestas en práctica son heredables. La clase **Shape** proporciona una interfaz heredable en la forma de tres funciones virtuales, que pueden ser contenidas por todos los miembros de la jerarquía. La clase **Shape** también proporciona algunas puestas en práctica, que podrán utilizar las clases derivadas en los primeros pocos niveles de la jerarquía.

Este estudio de caso hace énfasis en el hecho de que una clase puede heredar la interfaz y/o la puesta en práctica de una clase base. Las jerarquías diseñadas para la herencia de puestas en práctica tienden a tener su funcionalidad alta dentro de la jerarquía. Las jerarquías diseñadas para la herencia de interfaz tienden a tener su funcionalidad baja dentro de la jerarquía.

La clase base **Shape** (figura 20.2, parte 1) está formada de tres funciones virtuales públicas y no contiene ningún dato. La función **printShapeName** es virtual pura, por lo que es redefinida en cada una de las clases derivadas. Las funciones **area** y **volume** se definen para devolver 0.0. Estas funciones se redefinen en las clases derivadas cuando es apropiado para dichas clases el tener un cálculo distinto para **area** y/o un cálculo distinto de **volume**.


```
// SHAPE.H
// Definition of abstract base class Shape
#ifndef SHAPE_H
#define SHAPE_H

class Shape {
public:
    virtual float area() const { return 0.0; }
    virtual float volume() const { return 0.0; }
    virtual void printShapeName() const = 0; // pure virtual
};

#endif
```

Fig. 20.2 Definición de la clase base abstracta **Shape** (parte 1 de 9).

La clase **Point** (figura 20.2 partes 2 y 3) es derivada de **Shape** con herencia pública. Un **Point** no tiene ni área ni volumen, por lo que las funciones miembro de clase base **area** y **volume** aquí no se redefinen —se heredan tal y como están definidas en **Shape**. La función **printShapeName** es una puesta en práctica de una función virtual, que en la clase base fue definida como una virtual pura. Otras funciones miembro incluyen una función **set**, a fin de asignar coordenadas nuevas **x** y **y** a un **Point** y función **get**, para regresar las coordenadas **x** y **y** de un **Point**.

La clase **Circle** (figura 20.2 partes 4 y 5) es derivada de **Point** con herencia pública. Un **Circle** no tiene volumen, por lo que la función miembro de clase base **volume** no se redefine aquí —es heredada de **Shape** (y subsecuentemente en **Point**). Un **Circle** tiene área, por lo que en esta clase se redefine la función **area**. La función **printShapeName** es una puesta en

```
// POINT1.H
// Definition of class Point
#ifndef POINT1_H
#define POINT1_H
#include <iostream.h>
#include "shape.h"

class Point : public Shape {
    friend ostream &operator<<(ostream &, const Point &);
public:
    Point(float = 0, float = 0); // default constructor
    void setPoint(float, float);
    float getX() const { return x; }
    float getY() const { return y; }
    virtual void printShapeName() const { cout << "Point: "; }
private:
    float x, y; // x and y coordinates of Point
};

#endif
```

Fig. 20.2 Definición de clase **Point** (parte 2 de 9).

```
// POINT1.CPP
// Member function definitions for class Point
#include <iostream.h>
#include "point1.h"

Point::Point(float a, float b)
{
    x = a;
    y = b;
}

void Point::setPoint(float a, float b)
{
    x = a;
    y = b;
}

ostream &operator<<(ostream &output, const Point &p)
{
    output << '[' << p.x << ", " << p.y << '];'

    return output; // enables concatenated calls
}
```

Fig. 20.2 Definiciones de función miembro para la clase **Point** (parte 3 de 9).

```
// CIRCLE1.H
// Definition of class Circle
#ifndef CIRCLE1_H
#define CIRCLE1_H
#include "point1.h"

class Circle : public Point {
    friend ostream &operator<<(ostream &, const Circle &);
public:
    // default constructor
    Circle(float r = 0.0, float x = 0.0, float y = 0.0);

    void setRadius(float);
    float getRadius() const;
    virtual float area() const;
    virtual void printShapeName() const { cout << "Circle: "; }
private:
    float radius; // radius of Circle
};

#endif
```

Fig. 20.2 Definición de clase **Circle** (parte 4 de 9).

práctica de una función virtual, que en la clase base fue definida como una virtual pura. Si esta función no se redefine aquí, sería heredada la versión **Point** de la función.

```
// CIRCLE1.CPP
// Member function definitions for class Circle
#include <iostream.h>
#include <iomanip.h>
#include "circle1.h"

// Constructor for Circle call constructor for Point
Circle::Circle(float r, float a, float b)
    : Point(a, b) // call base-class constructor
{ radius = r > 0 ? r : 0; }

// Set radius
void Circle::setRadius(float r) { radius = r > 0 ? r : 0; }

// Get radius
float Circle::getRadius() const { return radius; }

// Calculate area of a Circle
float Circle::area() const { return 3.14159 * radius * radius; }

// Output a circle in the form: Center=[x, y]; Radius=###
ostream &operator<<(ostream &output, const Circle &c)
{
    output << '[' << c.getX() << ", " << c.getY()
        << "]; Radius = " << setiosflags(ios::showpoint)
        << setprecision(2) << c.radius;

    return output; // enables concatenated calls
}
```

Fig. 20.2 Definiciones de función miembro para la clase `Circle` (parte 5 de 9).

Otras funciones miembro incluyen una función `set`, para asignar un nuevo `radius` a un `Circle` y una función `get`, para devolver `radius` de un `Circle`.

La clase `Cylinder` (figura 20.2 partes 6 y 7) es derivada de `Circle` con herencia pública. Un `Cylinder` tiene área y volumen, por lo que las funciones `area` y `volume` ambas se redefinen en esta clase. La función `printShapeName` es una puesta en práctica de una función virtual, que en la clase base fue definida como virtual pura. Si aquí no se redefine esta función, sería heredada la versión `Circle` de la función. Otras funciones miembro incluyen una función `set`, para asignar un nuevo `radius` a un `Circle` y una función `get`, para devolver `radius` de un `Circle`.

El programa manejador (figura 20.2 partes 8 y 9) empieza produciendo el objeto `Point` de la clase `point`, el objeto `circle` de la clase `Circle` y el objeto `cylinder` de clase `Cylinder`. Se invoca la función `printShapeName` para cada uno de los objetos y cada objeto es extraído con su operador de inserción de flujo homónimo, a fin de ilustrar que los objetos han sido inicializados en forma correcta. A continuación, se declara el apuntador `ptr` del tipo `Shape *`. Se utiliza este apuntador para señalar a cada uno de los objetos de la clase derivada. Primero se asigna la dirección de `point` a `ptr` y se efectúan las siguientes llamadas

```
ptr->printShapeName()
ptr->area()
ptr->volume()
```

```
// CYLINDER.H
// Definition of class Cylinder
#ifndef CYLINDER_H
#define CYLINDER_H
#include "circle1.h"

class Cylinder : public Circle {
    friend ostream &operator<<(ostream &, const Cylinder &);
public:
    // default constructor
    Cylinder(float h = 0.0, float r = 0.0,
            float x = 0.0, float y = 0.0);

    void setHeight(float);
    virtual float area() const;
    virtual float volume() const;
    virtual void printShapeName() const { cout << "Cylinder: "; }
private:
    float height; // height of Cylinder
};

#endif
```

Fig. 20.2 Definición de clase `Cylinder` (parte 6 de 9).

para invocar estas funciones para el objeto al cual apunta `ptr`. La salida ilustra que las funciones han sido de forma correcta invocadas para `point`— “`Point:` ” es extraído y el área y el volumen son ambos `0.00`. A continuación, se asigna a `ptr` la dirección del objeto `circle` y se invocan las mismas funciones. La salida ilustra que las funciones han sido correctamente invocadas para el objeto `circle` se extrae— “`Circle:` ” el área de `circle` se calcula, y el volumen es `0.00`. Por último, `ptr` se asigna a la dirección del objeto `cylinder` y se invocan las mismas funciones. La salida ilustra que las funciones han sido invocadas correctamente para el objeto `cylinder`, se extrae `cylinder`— “`Cylinder:` ” se calcula el área de `cylinder` y se calcula el volumen de `cylinder`. Todas las llamadas de función `printShapeName`, `area` y `volume` se resuelven en tiempo de ejecución mediante ligaduras dinámicas.

Resumen

- Con las funciones virtuales y el polimorfismo, se hace posible diseñar y poner en práctica sistemas que son más extensibles. Se pueden escribir programas para procesar objetos de tipos que pudieran no existir cuando el programa está aún en desarrollo.
- Las funciones virtuales y la programación polimórfica pueden eliminar la necesidad de la lógica `switch`. El programador puede utilizar el mecanismo de las funciones virtuales para llevar a cabo de manera automática la lógica equivalente y, por lo tanto, evitando los tipos de errores típicamente asociados con la lógica `switch`. Un código cliente que tome decisiones sobre tipos de objetos y representaciones indicará un diseño de clase pobre.
- Se declara una función virtual haciendo preceder en la clase base el prototipo de función por la palabra reservada `virtual`.

```
// CYLINDR1.CPP
// Member function definitions for class Cylinder
#include <iostream.h>
#include <iomanip.h>
#include "cylindr1.h"

Cylinder::Cylinder(float h, float r, float x, float y)
    : Circle(r, x, y) // call base-class constructor
{ height = h > 0 ? h : 0; }

void Cylinder::setHeight(float h)
{ height = h > 0 ? h : 0; }

float Cylinder::area() const
{
    // surface area of Cylinder
    return 2 * Circle::area() +
        2 * 3.14159 * Circle::getRadius() * height;
}

float Cylinder::volume() const
{
    float r = Circle::getRadius();
    return 3.14159 * r * r * height;
}

ostream &operator<<(ostream &output, const Cylinder& c)
{
    output << '[' << c.getX() << ", " << c.getY()
        << "]; Radius = " << setiosflags(ios::showpoint)
        << setprecision(2) << c.getRadius()
        << "; Height = " << c.height;

    return output; // enables concatenated calls
}
```

Fig. 20.2 Definiciones de función miembro para la clase `Cylinder` (parte 7 de 9).

- Si así lo desean, las clases derivadas pueden proveer sus propias puestas en práctica de una función virtual de clase base, pero si no es así, entonces se utilizará la puesta en práctica de la clase base.
- Si se llama una función virtual haciendo referencia a un objeto específico por su nombre y utilizando el operador de selección miembro punto, dicha referencia se resuelve en tiempo de compilación (esto se conoce como *ligadura estática*) y la función virtual llamada es aquella definida (o heredada) por la clase de dicho objeto particular.
- Existen muchas situaciones, sin embargo, en las cuales resulta útil definir clases para las cuales el programador no tiene jamás intención de producir objeto alguno. Estas clases se conocen como clases abstractas. Dado que en situaciones de herencia éstas son utilizadas como clases base, nos referiremos por lo general a ellas como clases base abstractas. No se pueden producir objetos de una clase base abstracta.

```
// FIG20_2.CPP
// Driver for point, circle, cylinder hierarchy
#include <iostream.h>
#include <iomanip.h>
#include "shape.h"
#include "point1.h"
#include "circle1.h"
#include "cylindr1.h"

main()
{
    // create some shape objects
    Point point(7, 11);
    Circle circle(3.5, 22, 8);
    Cylinder cylinder(10, 3.3, 10, 10);

    point.printShapeName(); // static binding
    cout << point << endl;

    circle.printShapeName(); // static binding
    cout << circle << endl;

    cylinder.printShapeName(); // static binding
    cout << cylinder << "\n\n";

    cout << setiosflags(ios::showpoint) << setprecision(2);
    Shape *ptr; // create base-class pointer

    // aim base-class pointer at derived-class Point object
    ptr = &point;
    ptr->printShapeName(); // dynamic binding
    cout << "x = " << point.getX() << "; y = " << point.getY()
        << "\nArea = " << ptr->area()
        << "\nVolume = " << ptr->volume() << "\n\n";

    // aim base-class pointer at derived-class Circle object
    ptr = &circle;
    ptr->printShapeName(); // dynamic binding
    cout << "x = " << circle.getX() << "; y = " << circle.getY()
        << "\nArea = " << ptr->area()
        << "\nVolume = " << ptr->volume() << "\n\n";

    // aim base-class pointer at derived-class Cylinder object
    ptr = &cylinder;
    ptr->printShapeName(); // dynamic binding
    cout << "x = " << cylinder.getX()
        << "; y = " << cylinder.getY()
        << "\nArea = " << ptr->area()
        << "\nVolume = " << ptr->volume() << endl;

    return 0;
}
```

Fig. 20.2 Manejador para la jerarquía punto, círculo y cilindro (parte 8 de 9).

```
Point: [7, 11]
Cicle: [22, 8]; Radius = 3.50
Cylinder: [10, 10]; Radius = 3.30; Height = 10.00

Point: x = 7.00; y = 11.00
Area = 0.00
Volume = 0.00

Cicle: x = 22.00; y = 8.00
Area = 38.48
Volume = 0.00

Cylinder: x = 10.00; y = 10.00
Area = 275.77
Volume = 342.12
```

Fig. 20.2 Manejador para la jerarquía punto, círculo y cilindro (parte 9 de 9).

- Las clases a partir de las cuales se pueden producir objetos se conocen como clases concretas.
- Una clase con funciones virtuales se puede convertir en abstracta al declarar como pura una o más de sus funciones virtuales. Una función virtual pura es una que tenga un inicializador de = 0 dentro de su declaración.
- Si una clase es derivada de una clase con una función virtual pura sin proporcionar en la clase derivada una definición para dicha función virtual pura, entonces dicha función virtual se conserva como pura en la clase derivada. Y en consecuencia, la clase derivada también es una clase abstracta.
- C++ permite el polimorfismo —la habilidad de los objetos de clases distintas, emparentados mediante la herencia, de responder en forma diferente a la misma llamada de función miembro.
- El polimorfismo se pone en práctica vía funciones virtuales.
- Cuando se hace una solicitud mediante un apuntador de clase base para el uso de una función virtual, C++ escoge la función redefinida correcta, en la clase derivada apropiada asociada con dicho objeto.
- Mediante el uso de las funciones virtuales y del polimorfismo, una llamada de función miembro puede causar distintas acciones, dependiendo del tipo de objeto que reciba la llamada.
- Aunque no podemos producir objetos de clases base abstractas, *podemos* declarar apuntadores a clases base abstractas. Estos apuntadores pueden entonces utilizarse para permitir manipulaciones polimórficas de objetos de clase derivada, cuando dichos objetos sean producidos a partir de clases concretas.
- Continuamente se están añadiendo nuevos tipos de clase a los sistemas. Se les da sitio a la nuevas clases mediante la ligadura dinámica (también llamada ligadura tardía). No es necesario conocer en tiempo de compilación el tipo de un objeto, para que una llamada de función virtual sea compilada. La llamada de función virtual se hace coincidir en tiempo de ejecución con la función miembro del objeto llamado.

- La ligadura dinámica permite que fabricantes independientes de software (ISV) distribuyan software sin tener que revelar secretos propietarios. Las distribuciones de software pueden estar sólo constituidas de archivos de cabecera y archivos objeto. No es necesario revelar código fuente. A partir de las clases proveídas por los ISV, los desarrolladores de software podrán entonces utilizar la herencia para derivar nuevas clases. Aquel software que funcione con las clases proveídas por los ISV, continuará funcionando con las clases derivadas y utilizará (vía la ligadura dinámica) las funciones virtuales redefinidas proporcionadas en esas clases.
- La ligadura dinámica requiere que, en tiempo de ejecución, se encamine la llamada a una función miembro virtual, a la versión de función virtual apropiada para dicha clase. Se pone en operación una tabla de función virtual, conocida como la vtable, como un arreglo que contiene apuntadores de función. Cada clase, que contenga funciones virtuales, tiene una vtable. Para cada función virtual dentro de la clase, la vtable tiene una entrada que incluye un apuntador de función a la versión de la función virtual a utilizar para un objeto de dicha clase. La función virtual a utilizar para una clase particular pudiera ser la función definida en dicha clase, o pudiera ser una función heredada, ya sea directa o indirecta de una clase base más alta, dentro de la jerarquía.
- Cuando una clase base proporciona una función miembro y la declara **virtual**, las clases derivadas pueden redefinir la función virtual, pero no están obligadas a hacerlo. Por lo tanto, una clase derivada puede utilizar la versión de clase base de un función miembro virtual y esto quedaría indicado en la vtable.
- Cada objeto de una clase con funciones virtuales contiene un apuntador a la vtable para dicha clase. Este apuntador no es accesible para el programador. El apuntador de función apropiado para la vtable es obtenido y desreferenciado para completar la llamada en tiempo de ejecución. Esta búsqueda y desreferenciación de apuntadores en la vtable requiere de una sobrecarga nominal en tiempo de ejecución, usualmente menor que la del mejor código cliente posible.
- Declare el destructor de clase base virtual, si la clase contiene funciones virtuales. Esto hará virtuales a todos los destructores de clase derivada, aunque no tengan el mismo nombre que el destructor de clase base. Si un objeto en la jerarquía es explícitamente destruido mediante la aplicación del operador **delete** a un apuntador de clase base a un objeto de clase derivada, el destructor de la clase apropiada será llamado.

Terminología

clase base abstracta	ligadura temprana
clase abstracta	eliminación de enunciados switch
función de clase base virtual	conversión explícita de apuntadores
jerarquía de clase	extensibilidad
convertir objeto de clase derivada a objeto de clase base	conversión implícita de apuntador
convertir apuntador de clase derivada a apuntador de clase base	clase base indirecta
clase derivada	herencia
constructor de clase derivada	ligadura tardía
clase base directa	programación orientada a objetos (OOP)
ligadura dinámica	apuntador a una clase base
	apuntador a una clase derivada
	apuntador a una clase abstracta

polimorfismo	reutilización del software
función virtual pura (=0)	ligadura estática
función virtual redefinida	lógica switch
referencia a una clase base	destructor virtual
referencia a una clase derivada	función virtual
referencia a una clase abstracta	palabra reservada virtual

Errores comunes de programación

- 20.1 Resultará un error de sintaxis redefinir una clase derivada como función virtual de clase base, sin asegurarse que la función derivada tenga el mismo tipo de regreso y signatura que la versión de clase base.
- 20.2 Es un error de sintaxis intentar producir un objeto a partir de una clase abstracta (es decir, de una clase que contenga una o más funciones virtuales puras).
- 20.3 Declarar un constructor como función virtual. Los constructores no pueden ser virtuales.

Prácticas sanas de programación

- 20.1 A pesar que ciertas funciones en forma implícita son virtuales, en razón de una declaración hecha anteriormente en la jerarquía de clase, algunos programadores prefieren declarar estas funciones virtuales en forma explícita en cada uno de los niveles de la jerarquía, a fin de promover claridad en el programa.
- 20.2 Si una clase tiene funciones virtuales, incluya un destructor virtual, inclusive si para dicha clase no se requiere uno. Las clases que se deriven de esta clase pudieran contener destructores que deberán ser llamados en forma correcta.

Sugerencias de rendimiento

- 20.1 El polimorfismo, tal y como se pone en práctica mediante funciones virtuales y ligaduras dinámicas, resulta eficiente. Los programadores pueden utilizar estas capacidades con un impacto sólo nominal sobre el rendimiento del sistema.
- 20.2 Las funciones virtuales y las ligaduras dinámicas permiten la programación polimórfica, en contraste con la programación lógica **switch**. Los compiladores optimizantes de C++ por lo regular generan código que se ejecuta con una eficiencia por lo menos igual a la de la lógica basada en **switch** codificada manualmente.

Observaciones de ingeniería de software

- 20.1 Una consecuencia interesante del uso de funciones virtuales y polimorfismo es que los programas adquieren una apariencia simplificada. Contienen menos lógica de bifurcación, prefiriendo un código secuencial más sencillo. Esta simplificación facilita probar, depurar y mantener el programa.
- 20.2 Una vez declarada una función como virtual, se conserva como virtual a lo largo de toda la jerarquía de herencia, a partir de dicho punto.
- 20.3 Cuando una clase derivada decide no definir una función virtual, la clase derivada heredará la función virtual de la clase base inmediata.
- 20.4 Las funciones virtuales redefinidas deben tener el mismo tipo de regreso y la misma signatura que la función virtual base.
- 20.5 Si una clase se deriva de una clase con una función virtual pura, y para dicha función virtual pura no se ha dado definición en la clase derivada, entonces la función virtual también se conserva pura en la clase derivada. Por consecuencia, también la clase derivada será una clase abstracta.
- 20.6 Mediante las funciones virtuales y el polimorfismo, el programador puede ocuparse de generalidades y dejar que en tiempo de ejecución, el entorno se preocupe de lo específico. El programador

puede dirigir una amplia variedad de objetos haciendo que se comporten de formas apropiadas a dichos objetos incluso sin conocer los tipos de los mismos.

- 20.7 El polimorfismo fomenta la extensibilidad: software escrito para invocar comportamiento polimórfico se escribe en forma independiente del tipo de los objetos a los cuales los mensajes son enviados. Por lo tanto, nuevos tipos de objetos, que pudieran responder a mensajes existentes, pueden ser añadidos en dicho sistema sin modificar el sistema base. A excepción de la parte de código cliente que produce nuevos objetos, los programas no necesitan ser recompilados.
- 20.8 Una clase abstracta define una interfaz para los distintos miembros de una jerarquía de clase. La clase abstracta contiene funciones virtuales puras, que serán definidas en las clases derivadas. Mediante el polimorfismo todas las funciones en la jerarquía pueden utilizar esta misma interfaz.

Ejercicios de autoevaluación

- 20.1 Llene cada uno de los siguientes espacios vacíos:
 - a) Utilizando la herencia y el polimorfismo se ayuda a eliminar la lógica _____.
 - b) Una función virtual pura se define colocando una _____ al fin de su prototipo en la definición de clase.
 - c) Si una clase contiene una o más funciones virtuales puras, es una _____.
 - d) Una llamada de función resuelta en tiempo de compilación se conoce como ligadura _____.
 - e) Una llamada de función resuelta en tiempo de ejecución se conoce como ligadura _____.

Respuestas a los ejercicios de autoevaluación

- 20.1 a) **switch**. b) = 0. c) clase base abstracta. d) estática. e) dinámica.

Ejercicios

- 20.2 ¿Qué son las funciones virtuales? Describa alguna circunstancia en la cual las funciones virtuales serían apropiadas.
- 20.3 Dado que los constructores no pueden ser virtuales, describa algún procedimiento mediante el cual usted podría conseguir un efecto similar.
- 20.4 En el ejercicio 19.5 usted desarrolló una jerarquía de clase **Shape** y definió las clases dentro de la jerarquía. Modifique dicha jerarquía, de tal forma que la clase **Shape** sea una clase base abstracta que contenga la interfaz a la jerarquía. Derive **TwoDimensionalShape** y **ThreeDimensionalShape** de la clase **Shape** —estas clases también deberán ser abstractas. Utilice una función virtual **print** para extraer el tipo y dimensiones de cada clase. También incluya las funciones virtuales **area** y **volume** de tal forma que estos cálculos puedan ser ejecutados para objetos de cada clase concreta dentro de la jerarquía. Escriba un programa manejador que pruebe la jerarquía de clase **Shape**.
- 20.5 Utilizando la clase de plantilla **List** que se desarrolló en el ejercicio 17.10, cree una lista enlazada de apuntadores a objetos **Shape** (es decir, apuntadores a cualquier forma dentro de la jerarquía). Proporcione un operador de inserción de flujo homónimo para la clase **Shape**, que sólo llame la función virtual pura **print**. Utilice la capacidad de impresión de la lista enlazada para recorrer la lista y extraer cada objeto.

21

Flujo de entrada/salida de C++

Objetivos

- Comprender cómo utilizar el flujo de entrada/salida orientada a objetos de C++.
- Ser capaz de darle formato a las entradas y a las salidas.
- Comprender la jerarquía de clase de flujo de entradas/salidas.
- Comprender cómo introducir/extraer objetos de tipos definidos por usuario.
- Ser capaz de crear manipuladores de flujo definidos por usuario.
- Ser capaz de determinar el éxito o el fracaso de operaciones de entrada/salida.
- Ser capaz de ligar flujos de salida con flujos de entrada.

*Al parecer, el estar consciente... no está dividido en pedazos
...Las metáforas mediante las cuales quedaría descrito
con más naturalidad son "río" o "flujo".*
William James

Todas las noticias que merecen imprimirse.
Adolph S. Ochs

Sinopsis

- 21.1 Introducción
- 21.2 Flujos
 - 21.2.1 Archivos de cabecera de biblioteca iostream
 - 21.2.2 Clases y objetos del flujo de entradas/salidas
- 21.3 Salida de flujo
 - 21.3.1 Operador de inserción de flujo
 - 21.3.2 Cómo concatenar operadores de inserción/extracción de flujo
 - 21.3.3 Salida de variables Char *
 - 21.3.4 Extracción de caracteres mediante la función miembro put; cómo concatenar Puts
- 21.4 Entrada de flujo
 - 21.4.1 Operador de extracción de flujo
 - 21.4.2 Funciones miembro Get y Getline
 - 21.4.3 Otras funciones miembro istream (Peek, Putback, Ignore)
 - 21.4.4 Entradas/salidas de tipo seguro
- 21.5 Entradas/salidas sin formato utilizando Read, Gcount, y Write
- 21.6 Manipuladores de flujo
 - 21.6.1 Base de flujo integral: manipuladores de flujo Dec, Oct, Hex, y Setbase
 - 21.6.2 Precisión de punto flotante (Precision, Setprecision)
 - 21.6.3 Ancho de campo (Setw, Width)
 - 21.6.4 Manipuladores definidos por usuario
- 21.7 Estados de formato de flujo
 - 21.7.1 Banderas de estado de formato (Setf, Unsetf, Flags)
 - 21.7.2 Ceros a la derecha y puntos decimales (ios::showpoint)
 - 21.7.3 Justificación (ios::left, ios::right, ios::internal)
 - 21.7.4 Relleno (Fill, Setfill)
 - 21.7.5 Base de flujo integral (ios::dec, ios::oct, ios::hex, ios::showbase)
 - 21.7.6 Números de punto flotante; notación científica (ios::scientific, ios::fixed)
 - 21.7.7 Control de mayúsculas/minúsculas (ios::uppercase)
 - 21.7.8 Cómo activar y desactivar las banderas de formato (Flags, Setiosflags, Resetiosflags)
- 21.8 Estados de error de flujo
- 21.9 Entradas/salidas de tipos definidos por usuario
- 21.10 Cómo ligar un flujo de salida con un flujo de entrada

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Observaciones de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

21.1 Introducción

Las bibliotecas estándar de C++ proporcionan un conjunto extenso de capacidades de entrada/salida. Este capítulo analiza un conjunto de capacidades suficientes para llevar a cabo las operaciones de entradas y salidas más comunes y da un resumen general de las capacidades restantes.

Muchas de las características de entrada/salida descritas aquí son orientadas a objetos. El lector deberá encontrar de interés ver cómo dichas capacidades se ponen en práctica. Este estilo de entradas/salidas también hace mucho uso de características C++ como referencias, homonimia de funciones y homonimia de operadores.

Como veremos, C++ utiliza *entradas/salidas de tipo seguro*. Cada operación de entrada/salida se ejecuta de una manera que es sensible al tipo de los datos. Si una función de entrada/salida ha sido definida correctamente para manejar un tipo particular de datos, entonces dicha función es llamada en forma automática para manejar dicho tipo de datos. Si no existe coincidencia entre el tipo de los datos reales y una función para el manejo de dichos tipos de datos, se establece una indicación de error de compilación. Por lo tanto, no podrán introducirse datos equivocados dentro del sistema (como lo pueden hacer en C —una falla de C, que permite errores bastante sutiles y a menudo, extraños).

Los usuarios pueden especificar entradas/salidas de tipos definidos por usuario, así como de tipos estándar. Esta *extensibilidad* es una de las características más valiosas de C++.

Práctica sana de programación 21.1

En programas C++, utilice exclusivamente la forma de entradas/salidas de C++, a pesar del hecho que para los programadores C++ esté disponible el estilo de entradas/salidas de C.

Observación de ingeniería de software 21.1

El estilo de entradas/salidas de C++ es de tipo seguro.

Observación de ingeniería de software 21.2

C++ permite un tratamiento común de entradas/salidas de tipos predefinidos y de tipos definidos por usuario. Este tipo de estado común facilita el desarrollo de software en general y de la reutilización de software en particular.

21.2 Flujos

La entrada/salida en C++ ocurre en *flujos* de bytes. Un flujo es solo una secuencia de bytes. En operaciones de entrada, los bytes fluyen de un dispositivo (es decir, un teclado, una unidad de disco o una conexión de red) a la memoria principal. En operaciones de salida, los bytes fluyen de la memoria principal a un dispositivo (es decir, una pantalla de exhibición, una impresora, una unidad de disco o una conexión de red).

La aplicación asocia el significado con bytes. Los bytes pueden representar caracteres ASCII, datos en bruto de formato interno, imágenes gráficas, voz digitalizada, video digitalizado o cualquier otro tipo de información que pudiera una aplicación requerir.

El trabajo de los mecanismos de entrada y salida del sistema es mover los bytes de los dispositivos a la memoria y viceversa de forma consistente y confiable. Dichas transferencias a menudo, involucran movimientos mecánicos, como son el giro de un disco o de una cinta, o el tecleo en un teclado. El tiempo que, por lo general, toman esas transferencias es enorme en comparación con el tiempo que toma el procesador para la manipulación interna de los datos. Por lo tanto, para asegurar un rendimiento máximo, las operaciones de entrada/salida requieren una

cuidadosa planeación y sintonización. Temas como éste se discuten en detalle en los libros de texto de los sistemas operativos (De90).

C++ proporciona tanto capacidades de entradas y salidas de “bajo nivel” como de “alto nivel”. Las capacidades de entrada/salida de bajo nivel (es decir, entradas/salidas sin formato) especifican en forma típica que cierto número de bytes sólo deben ser transferidos del dispositivo a la memoria o de la memoria al dispositivo. En este tipo de transferencias, el byte individual es el elemento de interés. Estas capacidades de bajo nivel, de hecho proporcionan transferencias de alta velocidad y volumen, pero estas capacidades no son particularmente convenientes para las personas.

Las personas prefieren una visión de mayor nivel de las entradas/salidas, es decir, *entradas/salidas con formato*, en el cual los bytes están agrupados en unidades significativas como enteros, números de punto flotante, caracteres, cadenas y tipos definidos por usuario. Estas capacidades, orientadas al tipo, son satisfactorias para la mayor parte de las entradas/salidas que no correspondan al proceso de archivos de alto volumen.

Sugerencia de rendimiento 21.1

Utilice entradas/salidas sin formato para un rendimiento máximo en procesos de alto volumen de archivos.

21.2.1 Archivos de cabecera de biblioteca iostream

La biblioteca `iostream` de C++ proporciona cientos de capacidades de entrada/salida. Varios archivos de cabecera contienen porciones de la interfaz de la biblioteca.

La mayor parte de los programas de C++ deben incluir el archivo de cabecera `iostream.h`, que contiene información básica requerida para todas las operaciones de flujo de entrada/salida. El archivo de cabecera `iostream.h` contiene los objetos `cin`, `cout`, `cerr` y `clog` los cuales, como veremos, corresponden al flujo de entrada estándar, flujo de salida estándar y flujo de error estándar. Se proporcionan tanto capacidades de entrada y salida con, como sin formato.

El encabezado `iomanip.h` contiene información útil para llevar a cabo entradas/salidas con formato, mediante *manipuladores de flujo parametrizados*.

El encabezado `fstream.h` contiene información de importancia para llevar a cabo las operaciones de proceso de archivos controlados por el usuario.

El encabezado `strstream.h` contiene información de importancia para llevar a cabo *formato en memoria* (también conocido como *formato en núcleo*). Esto se parece al proceso de archivos, pero en las operaciones “entrada y salida” se ejecutan hacia y desde arreglos de caracteres, en vez de hacia y desde archivos.

El encabezado `stdiostream.h` contiene información de importancia para aquellos programas que combinen los estilos de C y de C++ para entradas/salidas. Los programas nuevos deberían evitar entradas/salidas de estilo C, pero aquellas personas que deben modificar programas en C existentes, o que transforman programas de C a C++, pueden encontrar útiles estas capacidades.

Cada puesta en práctica de C++ por lo general, contiene otras bibliotecas relacionadas con entradas/salidas que proporcionan capacidades específicas a cada sistema, como el control de dispositivos de uso especial para entradas/salidas de audio y de video.

21.2.2 Clases y objetos de flujo de entrada/salida

La biblioteca `iostream` contiene muchas clases para el manejo una gran variedad de operaciones de entradas/salidas. La clase `istream` apoya operaciones de entrada de flujo. La clase `ostream` apoya operaciones de salida de flujo. La clase `iostream` apoya tanto operaciones de entrada como de salida de flujo.

Las clases `istream` y `ostream` cada una de ellas han sido derivadas mediante herencia simple de la clase base `ios`. La clase `iostream` es una derivación mediante herencia múltiple, tanto de la clase `istream` como de la clase `ostream`. Estas relaciones de herencia se resumen en la figura 21.1.

La homonimia de operadores permite una forma de notación conveniente para llevar a cabo entradas/salidas. Se hace la homonimia del operador de desplazamiento a la izquierda (<<) para designar salida de flujo y se conoce como *operador de inserción de flujo*. Se hace la homonimia del operador de desplazamiento a la derecha (>>) a fin de designar entrada de flujo y se conoce como *operador de extracción de flujo*. Estos operadores son utilizadas con los objetos de flujo estándar `cin`, `cout`, `cerr` y `clog`, y con objetos de flujo definidos por usuario.

`cin` es un objeto de la clase `istream` y se dice que está “ligado a” (o conectado con) el dispositivo de entrada estándar, que por lo regular es el teclado. El operador de extracción de flujo, tal y como se usa en el siguiente enunciado, hace que se introduzca un valor para la variable entera `grade` desde `cin` a la memoria

```
cin >> grade;
```

Note que la operación de extracción de flujo es “lo suficiente inteligente” para “saber” cuál es el tipo de los datos. Suponiendo que `grade` ha sido declarado en forma correcta, no es necesario especificar ningún tipo adicional de información para utilizar el operador de extracción de flujo (como sería el caso, incidentalmente, en entradas/salidas en estilo C).

`cout` es un objeto de la clase `ostream` y se dice que está “ligado con” el dispositivo de salida estándar, que por lo regular es la pantalla de exhibición. El operador de inserción de flujo, tal y como se usa en el enunciado siguiente, hace que se extraiga el valor de la variable entera `grade` de la memoria hacia el dispositivo de salida estándar.

```
cout << grade;
```

Note que el operador de inserción de flujo es “lo suficiente inteligente” para “saber” el tipo de `grade` (suponiendo que éste haya sido declarado en forma correcta), por lo que no es necesario ningún tipo adicional de información para uso con el operador de inserción de flujo.

`cerr` es un objeto de la clase `ostream` y se dice que “está ligado con” el dispositivo de error estándar. Las salidas al objeto `cerr` no pasan por almacenamientos temporales o búfers. Esto significa que cada inserción a `cerr` hará que su salida aparezca de inmediato; esto resulta apropiado para una notificación instantánea de algún problema al usuario.

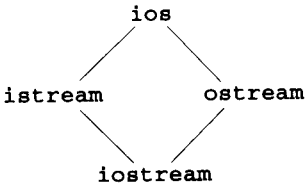


Fig. 21.1 Porción de la jerarquía de clase de flujo de entradas/salidas.

`clog` es un objeto de la clase `ostream` y también se dice que “está ligado con” el dispositivo de error estándar. Las salidas a `clog` sí pasan por almacenamiento temporal o búfer.

El procesamiento de archivo en C++ utiliza las clases `istream` para llevar a cabo operaciones de entrada de archivo, utiliza a `ofstream` para operaciones de salida de archivo, y `fstream` para operaciones de entrada/salida de archivo. La clase `istream` hereda de `istream`, la clase `ofstream` hereda de `ostream`, y la clase `fstream` hereda de `iostream`. Las relaciones de herencia de las clases relacionadas con entradas/salidas se resumen en la figura 21.2. En la jerarquía completa de clase de flujo de entradas/salidas en la mayor parte de las instalaciones existen muchas más clases incluidas, pero las clases que aquí se muestran proporcionan la gran mayoría de las capacidades que necesitarán la mayor parte de los programadores. Vea las referencias de biblioteca de clase correspondientes a su sistema C++ para más información de procesamiento de archivos.

21.3 Salida de flujo

La clase `ostream` de C++ da la capacidad para llevar a cabo salida con y sin formato. Las capacidades de salida incluyen: la salida de tipos de datos estándar mediante el operador de inserción de flujo; la salida de caracteres utilizando la función miembro `put`; salida sin formato mediante la función miembro `write` (sección 21.5); salida de enteros en formatos decimal, octal y hexadecimal (sección 21.6.1); salida de valores en punto flotante con distintas precisiones (sección 21.6.2), con puntos decimales obligados (21.7.2), en notación científica, y en notación fija (sección 21.7.6); salida de datos justificados en campos de anchos de campo designados (sección 21.7.3); salida de datos en campos rellenos de caracteres especiales (sección 21.7.4); y salida de letras mayúsculas en notación científica y hexadecimal (sección 21.7.7).

21.3.1 Operador de inserción de flujo

La salida de flujo puede ser ejecutada mediante el operador de inserción de flujo, es decir, el operador homónimo `<<`. Se hace la homonimia del operador `<<` para extraer elementos de datos de tipos incorporados, para extraer cadenas y para extraer valores de apuntadores. En la sección 21.9, veremos como hacer la homonimia de `<<` para extraer elementos de datos de tipos definidos por usuario. La figura 21.3 demuestra la salida de una cadena.

El ejemplo anterior utiliza un único enunciado de inserción de flujo a fin de mostrar la cadena de salida. Se pueden utilizar varios enunciados de inserción, como en la figura 21.4. Cuando este programa sea ejecutado, producirá la misma salida que el programa anterior.

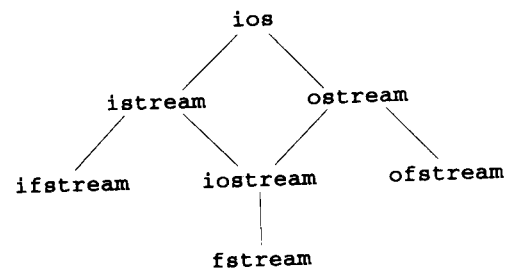


Fig. 21.2 Porción de la jerarquía de clase de flujo de entradas/salidas mostrando las clases de procesamiento de archivo clave.

```
// fig21_03.cpp
// Outputting a string using stream insertion.

#include <iostream.h>

main()
{
    cout << "Welcome to C++!\n";

    return 0;
}
```

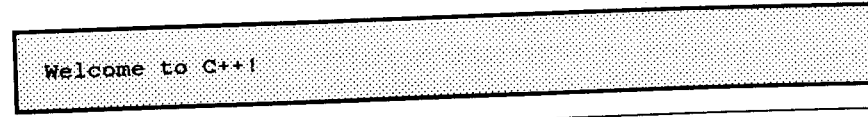


Fig. 21.3 Cómo extraer una cadena utilizando la inserción de flujo.

```
// fig21_04.cpp
// Outputting a string using two stream insertions.

#include <iostream.h>

main() ~
{
    cout << "Welcome to ";
    cout << "C++!\n";

    return 0;
}
```

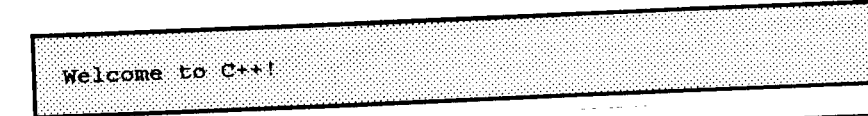


Fig. 21.4 Cómo extraer una cadena utilizando dos inserciones de flujo.

El efecto de la secuencia de escape `\n` (nueva línea) también se puede conseguir mediante el *manipulador de flujo* `endl` (terminar línea) como en la figura 21.5. El manipulador de flujo `endl` emite un carácter de nueva línea y, además, vacía el búfer de salida (es decir, hace que el búfer de salida sea extraído de inmediato, aunque todavía no esté lleno). El búfer de salida también puede ser vaciado, sólo mediante

```
cout << flush;
```

En la sección 21.6 se analizan en detalle los manipuladores de flujo. Las expresiones pueden ser extraídas tal y como se muestra en la figura 21.6.

Práctica sana de programación 21.2

Al extraer expresiones, colóquelas entre paréntesis, para evitar problemas de precedencia de operadores entre los operadores de la expresión y el operador `<<`.

```
// fig21_05.cpp
// Using the endl stream manipulator.
#include <iostream.h>

main()
{
    cout << "Welcome to ";
    cout << "C++!";
    cout << endl;    // end line stream manipulator

    return 0;
}
```

```
Welcome to C++!
```

Fig. 21.5 Cómo utilizar el manipulador de flujo `endl`.

```
// fig21_06.cpp
// Outputting expression values.
#include <iostream.h>

main()
{
    cout << "47 plus 53 is ";
    cout << 47 + 53;    // expression
    cout << endl;

    return 0;
}
```

```
47 plus 53 is 100
```

Fig. 21.6 Cómo extraer valores de expresiones.

21.3.2 Cómo concatenar operadores de inserción/extracción de flujo

Los operadores homónimos `<<` y `>>` cada uno de ellos puede ser utilizado en forma concatenada, tal y como se muestra en la figura 21.7.

Las múltiples inserciones de flujo de la figura 21.7 se ejecutan como si hubieran sido escritas:

```
((cout << "47 plus 53 is ") << 47 + 53) << endl;
```

Esto está permitido porque el operador homónimo `<<` devuelve una referencia a su objeto operando izquierdo, es decir, `cout`. Por lo tanto, la expresión entre paréntesis más a la izquierda

```
(cout << "47 plus 53 is ")
```

```
// fig21_07.cpp
// Concatenating the overloaded << operator.
#include <iostream.h>

main()
{
    cout << "47 plus 53 is " << 47 + 53 << endl;

    return 0;
}
```

```
47 plus 53 is 100
```

Fig. 21.7 Cómo concatenar el operador homónimo `<<`.

extrae la cadena especificada de caracteres y devuelve una referencia a `cout`. Esto permite que se evalúe la expresión entre paréntesis intermedia como

```
(cout << 47 + 53)
```

que extrae el valor entero `100` y devuelve una referencia a `cout`. A continuación se evalúa la expresión entre paréntesis más a la derecha como

```
cout << endl
```

que extrae una nueva línea, vacía a `cout` y devuelve una referencia a `cout`. Este último regreso no es utilizado.

21.3.3 Salida de variables `char*`

En entradas/salidas de estilo C, es necesario proporcionar información de tipo. C++ determina los tipos de datos en forma automática —lo que resulta una agradable mejoría respecto a C. Pero algunas veces esto “resulta un estorbo”. Por ejemplo, sabemos que una cadena de caracteres es del tipo `char*`. Suponga que deseamos imprimir el valor de dicho apuntador, es decir, la dirección en memoria del primer carácter de dicha cadena. Pero se ha hecho la homonimia del operador `<<` para imprimir datos del tipo `char*` como si fuera una cadena terminada por `null`. La solución es hacer una conversión explícita (cast) del apuntador a `void*` (esto debe ser hecho a cualquier variable de apuntador a la cual el programador desee extraer como una dirección). En la figura 21.8 se demuestra la impresión de una variable `char*` tanto en formato de cadena como de dirección. Note que la dirección se imprime como un número hexadecimal (de base 16). En C++ los números hexadecimales empiezan con `0x`, o `0X`. En las secciones 21.6.1, 21.7.4, 21.7.5 y 21.7.7 ampliamos la información respecto a cómo controlar las bases de los números.

21.3.4 Extracción de caracteres mediante la función miembro `put`; cómo concatenar `put`

La función miembro `put` extrae un carácter como en

```
cout.put('A');
```

```
// fig21_08.cpp
// Printing the address stored in a char* variable

#include <iostream.h>

main()
{
    char *string = "test";

    cout << "Value of string is: " << string
         << "\nValue of (void *)string is: "
         << (void *)string << endl;

    return 0;
}
```

```
Value of string is: test
Value of (void *)string is: 0x00aa
```

Fig. 21.8 Cómo imprimir la dirección almacenada en una variable `char *`.

que exhibe **A** en pantalla. Las llamadas a `put` pueden ser concatenadas como en

```
cout.put('A').put('\n');
```

que extrae **A**, seguida por un carácter de nueva línea. La función `put` también puede ser llamada con una expresión de valor ASCII, como en `cout.put(65)`, que también extrae a **A**.

21.4 Entrada de flujo

Consideremos ahora entradas de flujo. Esto se puede ejecutar con el operador de extracción de flujo, es decir, con el operador homónimo `>>`. Este operador por lo general, pasa por alto los *caracteres de espacio en blanco* (como espacios en blanco, tabuladores y nuevas líneas) existentes en el flujo de entrada. Más tarde veremos cómo modificar este comportamiento. El operador de extracción de flujo devuelve cero (falso) cuando dentro de un flujo encuentra el final de archivo. Cada flujo contiene un conjunto de *bits de estado* que se utilizan para controlar el estado del flujo (es decir, estados de formato, de establecimiento de errores, etcétera). La extracción de flujo hace que para entradas de tipo incorrecto se active **failbit** del flujo, y si la operación falla hace que se active **badbit** del flujo. Veremos pronto cómo probar estos bits después de una operación de entrada/salida. En las secciones 21.7 y 21.8 se analizan los bits de estado de flujo en detalle.

21.4.1 Operador de extracción de flujo

Para leer dos enteros, utilice el objeto `cin` y el operador de extracción de flujo `>>` homónimo, tal y como aparece en la figura 21.9.

La relativamente alta precedencia de los operadores `>>` y `<<` pueden causar problemas. Por ejemplo, el programa de la figura 21.10 no se compilará en forma correcta sin los paréntesis que encierran la expresión condicional. El lector deberá de verificar lo anterior.

```
// fig21_09.cpp
// Calculating the sum of two integers
// input from the keyboard with cin
// and the stream extraction operator.
#include <iostream.h>

main()
{
    int x, y;

    cout << "Enter two integers: ";
    cin >> x >> y;
    cout << "Sum of " << x << " and " << y << " is: "
         << x + y << endl;

    return 0;
}
```

```
Enter two integers: 30 92
Sum of 30 and 92 is: 122
```

Fig. 21.9 Cómo calcular la suma de dos enteros introducidos desde el teclado mediante `cin` y el operador de extracción de flujo.

```
// fig21_10.cpp
// Revealing a precedence problem.
// Need parentheses around the conditional expression.
#include <iostream.h>

main()
{
    int x, y;

    cout << "Enter two integers: ";
    cin >> x >> y;
    cout << x << (x == y ? " is" : " is not")
         << " equal to " << y << endl;

    return 0;
}
```

```
Enter two integers: 7 5
7 is not equal to 5
```

```
Enter two integers: 8 8
8 is equal to 8
```

Fig. 21.10 Cómo evitar un problema de precedencia entre el operador de inserción de flujo y el operador condicional.

Error común de programación 21.1

Intentar leer de un ostream (o de cualquier otro flujo de solo salida).

Error común de programación 21.2

Intentar escribir a un istream (o a cualquier otro flujo de sólo entrada).

Error común de programación 21.3

Omitir paréntesis para obligar a una correcta precedencia al utilizar los operadores de inserción de flujo << o de extracción de flujo >> que tienen una relativamente alta precedencia.

Una forma popular para introducir una serie de valores es utilizar la operación de extracción de flujo en el encabezado de un ciclo **while**. La extracción devuelve falso (cero) cuando encuentra el fin de archivo. Considere el programa de la figura 21.11, mismo que encuentra la calificación más alta de un examen. Suponga que el número de calificaciones no es conocido con anticipación, y que el usuario escribirá un fin de archivo, a fin de indicar que todas las calificaciones han sido introducidas. La condición **while (cin >> grade)**, se convierte en cero (es decir, falso) cuando el usuario introduzca el fin de archivo.

```
// fig21_11.cpp
// Stream extraction operator returning
// false on end-of-file.
#include <iostream.h>

main()
{
    int grade, highestGrade = -1;
    cout << "Enter grade (enter end-of-file to end): ";
    while (cin >> grade) {
        if (grade > highestGrade)
            highestGrade = grade;
        cout << "Enter grade (enter end-of-file to end): ";
    }
    cout << "\nHighest grade is: " << highestGrade
        << endl;
    return 0;
}
```

```
Enter grade (enter end-of-file to end): 67
Enter grade (enter end-of-file to end): 87
Enter grade (enter end-of-file to end): 73
Enter grade (enter end-of-file to end): 95
Enter grade (enter end-of-file to end): 34
Enter grade (enter end-of-file to end): 99
Enter grade (enter end-of-file to end): ^Z

Highest grade is: 99
```

Fig. 21.11 El operador de extracción de flujo devolviendo falso al fin de archivo.

21.4.2 Funciones miembro get y getline

La función miembro **get** sin ningún argumento, introduce un carácter del flujo designado (incluso si se trata de un espacio en blanco) y devuelve dicho carácter como el valor de la llamada de función. Esta versión de **get** devuelve **EOF** cuando en el flujo se encuentra el fin de archivo. **EOF** por lo regular es **-1**, a fin de distinguirlo de valores de caracteres ASCII (esto de un sistema a otro pudiera variar).

En la figura 21.12 se demuestra el uso de las funciones miembro **eof** y **get** en el flujo de entrada **cin** y de la función miembro **put** en el flujo de salida **cout**. El programa primero imprime el valor de **cin.eof()**, es decir, 0, (falso) a fin de mostrar que en **cin** no ha ocurrido el fin de archivo. El usuario escribe una línea de texto, seguida por un fin de archivo (**<ctrl>-z** seguido por **<return>**, en sistemas IBM PC y compatibles, y **<ctrl>-d**, en sistemas UNIX y Macintosh). El programa leerá cada uno de los caracteres y mediante la función miembro **put** lo extraerá a **cout**. Cuando se encuentra con el fin de archivo, se termina el **while**, y **cin.eof()** —ahora **1** (verdadero)— se vuelve a imprimir, a fin de demostrar que en **cin** se ha definido el fin de archivo. Note que este programa utiliza la versión de la función miembro **get** de **istream**, que no toma argumentos y que devuelve el carácter que se está introduciendo.

```
// fig21_12.cpp
// Using member functions get, put, and eof.

#include <iostream.h>

main()
{
    int c;

    cout << "Before input, cin.eof() is "
        << cin.eof() << '\n'
        << "Enter a sentence followed by end-of-file:\n";

    while ((c = cin.get()) != EOF)
        cout.put(c);

    cout << "\nAfter input, cin.eof() is "
        << cin.eof() << endl;

    return 0;
}
```

```
Before input, cin.eof() is 0
Enter a sentence followed by end-of-file:
Testing the get and put member functions^Z
Testing the get and put member functions
After input cin.eof() is 1
```

Fig. 21.12 Cómo utilizar las funciones miembro **get**, **put** y **eof**.

La función miembro `get` con un argumento de carácter, introduce el carácter siguiente del flujo de entrada (aun si se trata de un carácter en blanco). Esta versión de `get` devuelve falso cuando se encuentra el fin de archivo; de lo contrario esta versión de `get` devuelve una referencia al objeto `istream` para la cual se ha invocado a la función miembro `get`.

Una tercera versión de la función miembro `get` toma tres argumentos —un arreglo de caracteres, un límite de tamaño y un delimitador (con un valor por omisión de `'\n'`). Esta versión lee caracteres a partir del flujo de entrada. Lee hasta uno menos que el número máximo especificado de caracteres y termina, o termina en cuanto lea el delimitador. Se inserta un carácter nulo para terminar la cadena de caracteres, en el arreglo de caracteres que se utiliza como búfer por el programa. El delimitador no es colocado en el arreglo de caracteres, sino que es conservado en el flujo de entrada. En la figura 21.13 se compara la entrada utilizando `cin` con la extracción de flujo y la entrada con `cin.get`.

```
// fig21_13.cpp
// Contrasting input of a string with cin and cin.get.

#include <iostream.h>

const int SIZE = 80;

main()
{
    char buffer1[SIZE], buffer2[SIZE];

    cout << "Enter a sentence:\n";
    cin >> buffer1;
    cout << "\nThe string read with cin was:\n"
         << buffer1 << "\n\n";

    cin.get(buffer2, SIZE);
    cout << "The string read with cin.get was:\n"
         << buffer2 << endl;

    return 0;
}
```

```
Enter a sentence:
Contrasting string input with cin and cin.get

The string read with cin was:
Contrasting

The string read with cin.get was:
string input with cin and cin.get
```

Fig. 21.13 Comparación de entradas de una cadena mediante `cin`, con la extracción de flujo y la entrada mediante `cin.get`.

La función miembro `getline` opera como la tercera versión de la función miembro `get` e inserta un carácter nulo después de la línea en el arreglo de caracteres. La función `getline` elimina el delimitador del flujo, pero no lo almacena en el arreglo de caracteres. El programa de la figura 21.14 demuestra el uso de la función miembro `getline` para la introducción de una línea de texto.

21.4.3 Otras funciones miembro `istream` (`peek`, `putback`, `ignore`)

La función miembro `ignore` pasa por alto un número designado de caracteres (un carácter por omisión) o da por terminado, al encontrar un delimitador designado (el delimitador por omisión es `EOF`).

La función miembro `putback` coloca el carácter previo obtenido por un `get` del flujo de entrada de regreso en dicho flujo. Esta función es útil para aquellas aplicaciones que rastrean un flujo de entrada buscando un campo que se inicie con un carácter específico. Cuando se introduce dicho carácter, la aplicación devuelve este carácter al flujo, a fin de que el carácter pueda ser incluido en los datos que serán introducidos.

La función miembro `peek` devuelve el carácter siguiente de un flujo de entrada, pero sin retirarlo del flujo.

```
// fig21_14.cpp
// Character input with member function getline.

#include <iostream.h>

const SIZE = 80;

main()
{
    char buffer[SIZE];

    cout << "Enter a sentence:\n";
    cin.getline(buffer, SIZE);

    cout << "\nThe sentence entered is:\n"
         << buffer << endl;

    return 0;
}
```

```
Enter a sentence:
Using the getline member function

The sentence entered is:
Using the getline member function
```

Fig. 21.14 Entrada de caracteres mediante la función miembro `getline`.

21.4.4 Entradas/salidas de tipo seguro

C++ ofrece *entradas/salidas de tipo seguro*. Se hace la homonimia de los operadores << y >> a fin de que acepten elementos de datos de tipos específicos. Si se procesan datos no esperados, se establecen varias banderas de error, mismas que el usuario puede probar, para determinar si una operación de entrada/salida ha tenido éxito o ha fracasado. De esta manera el programa “se conserva en control”. Analizaremos estas banderas de error en la sección 21.8.

21.5 Entradas/salidas sin formato utilizando read, gcount, y write

Se lleva a cabo *entradas/salidas sin formato* mediante las funciones miembro **read** y **write**. Cada una de estas funciones introduce o extrae cierta cantidad de bytes de o desde un arreglo de caracteres existente en memoria. Estos bytes no contienen ningún tipo de formato. Sólo son introducidos o extraídos como bytes en bruto. Por ejemplo, la llamada

```
char buffer[] = "HAPPY BIRTHDAY";
cout.write(buffer, 10);
```

extrae los primeros 10 bytes de **buffer**. Dado que la cadena de caracteres se evalúa a la dirección del primero de los caracteres, la llamada

```
cout.write("ABCDEFGHIJKLMNOPQRSTUVWXYZ", 10);
```

exhibe los primeros 10 caracteres del alfabeto.

La función miembro **read** introduce un número designado de caracteres en un arreglo de caracteres. Si son leídos menos que el número designado de caracteres, se activa **failbit**. Pronto veremos cómo determinar si **failbit** ha sido activado (vea la sección 21.8).

En la figura 21.5 se demuestra el uso de las funciones miembro **read** y **gcount** de **istream**, y la función miembro **write** de **ostream**. El programa introduce 20 caracteres, (a partir de una secuencia de entrada más larga) en el arreglo de caracteres de nombre **buffer** mediante **read**, determina el número de caracteres introducidos mediante **gcount**, y extra los caracteres existentes en **buffer** mediante **write**

21.6 Manipuladores de flujo

C++ proporciona varios *manipuladores de flujo* que ejecutan tareas de formato. Los manipuladores de flujo ofrecen capacidades tales como definición de anchos de campo, definición de precisiones, establecimiento o eliminación de banderas de formato, establecimiento de caracteres de relleno en campos, vaciado de flujos, inserción de nueva línea en el flujo de salida y vaciado de flujo, inserción de un carácter nulo en el flujo de salida, y pasar por alto los espacios en blanco del flujo de entrada. Estas características se describen en las secciones siguientes.

21.6.1 Base de flujo integral: manipuladores de flujo dec, oct, hex, y setbase

Los enteros por lo general, se interpretan como valores decimales (de base 10). Para cambiar la base con la cual se interpretan los enteros dentro de un flujo, inserte el manipulador **hex** para definir la base a hexadecimal (de base 16), o el manipulador **oct** para definir a base octal (de base 8). Inserte el manipulador de flujo **dec** para devolver la base del flujo a decimal.

```
// fig21_15.cpp
// Unformatted I/O with the read,
// gcount and write member functions.

#include <iostream.h>

const int SIZE = 80;

main()
{
    char buffer[SIZE];

    cout << "Enter a sentence:\n";
    cin.read(buffer, 20);
    cout << "\nThe sentence entered was:\n";
    cout.write(buffer, cin.gcount());
    cout << endl;

    return 0;
}
```

```
Enter a sentence:
Using the read, write, and gcount member functions

The sentence entered was:
Using the read, writ
```

Fig. 21.15 Entradas/salidas sin formato mediante las funciones miembro **read**, **gcount** y **write**.

La base de un flujo también puede modificarse mediante el manipulador de flujo **setbase**, que toma un argumento entero 10, 8 ó 16 para definir la base. Dado que **setbase** toma un argumento, se conoce como un *manipulador de flujo parametrizado*. El uso de **setbase** o de cualquier otro manipulador parametrizado requiere de la inclusión del archivo de cabecera **iosmanip.h**. La base de flujo se conservará sin cambios hasta que sea modificada en forma explícita. En la figura 21.16 se muestra la utilización de los manipuladores de flujo **hex**, **oct**, **dec** y **setbase**.

21.6.2 Precisión de punto flotante (precision, setprecision)

Podemos controlar la *precisión* de los números en punto flotante, es decir, el número de dígitos a la derecha del punto decimal, ya sea mediante el manipulador de flujo **setprecision** o la función miembro **precision**. Una llamada a cualquiera de estas funciones define la precisión para todas las operaciones subsecuentes de salida, hasta la siguiente llamada definidora de precisión. La función miembro **precision** sin argumento, devuelve el ajuste actual de precisión. El programa de la figura 21.17 utiliza tanto la función miembro **precision**, como el manipulador **setprecision**, para imprimir una tabla mostrando la raíz cuadrada de 2 con precisiones variando desde 0 hasta 9. Note que la precisión 0 tiene un significado especial. Restaura la *precisión por omisión* de valor 6.

```
// fig21_16.cpp
// Using hex, oct, dec and setbase stream manipulators.
#include <iostream.h>
#include <iomanip.h>

main()
{
    int n;

    cout << "Enter a decimal number: ";
    cin >> n;

    cout << n << " in hexadecimal is: "
        << hex << n << '\n'
        << dec << n << " in octal is: "
        << oct << n << '\n'
        << setbase(10) << n << " in decimal is: "
        << n << endl;

    return 0;
}
```

```
Enter a decimal number: 20
20 in hexadecimal is: 14
20 in octal is: 24
20 in decimal is: 20
```

Fig. 21.16 Cómo utilizar los manipuladores de flujo hex, oct, dec, y setbase.

21.6.3 Ancho de campo (Setw, Width)

La función miembro **width** define el ancho de campo y devuelve el ancho anterior. Si los valores procesados son menores que el ancho de campo, se insertan *caracteres de llenado* como *relleno*. Un valor más ancho que el ancho designando no será truncado —se imprimirá el número completo. El ajuste de ancho se aplica sólo a la siguiente inserción o extracción; después y de forma implícita el ancho se define a 0, es decir, los valores de salida sencillamente serán tan anchos como lo requieran. La función **width** sin argumento devuelve el ajuste presente.

Error común de programación 21.4

Cuando no se de un campo lo suficiente amplio para manejar salidas, dichas salidas se imprimirán del ancho que requieran, posiblemente causando salidas erróneas o difíciles de leer.

En la figura 21.18 se demuestra el uso de la función miembro **width** tanto en la entrada como en la salida. Note que en el caso de la entrada, se leerá un máximo de un carácter menos que el ancho establecido, porque se deja lugar para el carácter nulo a colocarse en la cadena de entrada. Recuerde que la extracción de flujo se da por terminada cuando se encuentra un espacio en blanco a la derecha. El manipulador de flujo **setw** también puede ser utilizado para definir el ancho de campo.

```
// fig21_17.cpp
// Controlling precision of floating point values
#include <iostream.h>
#include <iomanip.h>
#include <math.h>

main()
{
    double root2 = sqrt(2.0);

    cout << "Square root of 2 with precisions 0-9.\n"
        << "Precision set by the "
        << "precision member function:\n";

    for (int places = 0; places <= 9; places++) {
        cout.precision(places);
        cout << root2 << endl;
    }

    cout << "\nPrecision set by the "
        << "setprecision manipulator:\n";

    for (places = 0; places <= 9; places++)
        cout << setprecision(places) << root2 << endl;

    return 0;
}
```

```
Square root of 2 with precisions 0-9.
Precision set by the precision member function:
1.414214
1.4
1.41
1.414
1.4142
1.41421
1.414214
1.4142136
1.41421356
1.414213562

Precision set by the setprecision manipulator:
1.414214
1.4
1.41
1.414
1.4142
1.41421
1.414214
1.4142136
1.41421356
1.414213562
```

Fig. 21.17 Cómo controlar la precisión de valores de punto flotante.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE

```
// fig21_18.cpp
// Demonstrating the width member function

#include <iostream.h>

main()
{
    int w = 4;
    char string[10];

    cout << "Enter a sentence:\n";
    cin.width(5);

    while (cin >> string) {
        cout.width(w++);
        cout << string << '\n';
        cin.width(5);
    }

    return 0;
}
```

```
Enter a sentence:
This is a test of the width member function^Z
This
  is
    a
  test
    of
  the
  width
    h
  memb
    er
  func
  tion
```

Fig. 21.18 Demostración de la función miembro `width`.

21.6.4 Manipuladores definidos por usuario

Los usuarios pueden crear sus propios manipuladores de flujo. En la figura 21.19 se muestra la creación y el uso de los nuevos manipuladores de flujo `bell`, `ret` (retorno de carro), `tab` y `endLine`. Los usuarios también pueden crear sus propios manipuladores de flujo parametrizados —consulte sus manuales de instalación para encontrar las instrucciones de cómo llevar a cabo lo anterior.

21.7 Estados de formato de flujo

Varias *banderas de formato* especifican el tipo de formato a llevarse a cabo durante las operaciones de entrada/salida de flujo. Las funciones miembro `setf`, `unsetf` y `flags` controlan los ajustes de las banderas.

```
// fig21_19.cpp
// Testing user-defined, nonparameterized manipulators

#include <iostream.h>

// bell manipulator (using escape sequence \a)
ostream& bell(ostream& output)
{
    return output << '\a';
}

// ret manipulator (using escape sequence \r)
ostream& ret(ostream& output)
{
    return output << '\r';
}

// tab manipulator (using escape sequence \t)
ostream& tab(ostream& output)
{
    return output << '\t';
}

// endLine manipulator (using escape sequence \n
// and the flush member function)
ostream& endLine(ostream& output)
{
    return output << '\n' << flush;
}

main()
{
    cout << "Testing the tab manipulator:\n"
        << 'a' << tab << 'b' << tab << 'c' << endLine
        << "Testing the ret and bell manipulators:\n"
        << ".....";

    for (int i = 1; i <= 100; i++)
        cout << bell;

    cout << ret << "-----" << endLine;

    return 0;
}
```

```
Testing the tab manipulator:
a      b      c
Testing the ret and bell manipulators:
-----
```

Fig. 21.19 Cómo probar manipuladores definidos por usuario no parametrizados.

21.7.1 Banderas de estado de formato (setf, unsetf, flags)

Las banderas de estado de formato mostradas en la figura 21.20 se definen como una enumeración en la clase `ios` y son explicadas en la siguientes varias secciones.

Estas banderas pueden ser controladas por las funciones miembro `flags`, `setf` y `unsetf`, pero muchos programadores de C++ prefieren utilizar manipuladores de flujo (vea la sección 10.7.8). El programador puede utilizar la operación "OR a nivel de bits" `|`, para combinar varias opciones en un solo valor `int` (vea la figura 10.23). Llamar la función miembro `flags` para un flujo y especificar estas opciones de tipo "OR" define las opciones sobre dicho flujo y devuelve un valor que contiene las opciones anteriores. A menudo, este valor es guardado de tal forma que con este valor guardado pueda llamarse a `flags`, a fin de restaurar las opciones de flujo anteriores.

La función `flags` debe especificar un valor que represente los ajustes de todas las banderas. La función `setf` de un argumento, por otra parte, define una o más banderas "operadas con OR" o bien las opera con "OR" con los ajustes de banderas existentes a fin de formar un nuevo estado de formato.

El manipulador de flujo parametrizado `setiosflags` ejecuta las mismas funciones que la función `setf`. El manipulador de flujo `resetiosflags` lleva a cabo las mismas funciones que la función miembro `unsetf`. Para utilizar cualquiera de estos manipuladores de flujo, asegúrese de incluir `#include <iomanip.h>`.

La bandera `skipws` indica que `>>` deberá pasar por alto los espacios en blanco de un flujo de entrada. El comportamiento por omisión de `>>` es precisamente pasar por alto los espacios en blanco. Para modificar lo anterior, utilice la llamada `unsetf (ios::skipws)`. El manipulador de flujo `ws` pudiera también ser utilizado para definir que deben de saltarse los espacios en blanco.

21.7.2 Ceros a la derecha y puntos decimales (ios::showpoint)

La bandera `showpoint` se utiliza para obligar a un número de punto flotante a que se extraiga con punto decimal y ceros a la derecha. Sin la definición de `showpoint` un valor de punto flotante de `79.0` sería impreso como `79` y si `showpoint` estuviera definido sería impreso como

```
ios::skipws
ios::left
ios::right
ios::internal
ios::dec
ios::oct
ios::hex
ios::showbase
ios::showpoint
ios::uppercase
ios::showpos
ios::scientific
ios::fixed
ios::unitbuf
ios::stdio
```

Fig. 21.20 Banderas de estado de formato.

`79.000000` (o con tantos ceros a la derecha como estuvieran especificados en la posición actual). El programa de la figura 21.21 muestra el uso de la función miembro `setf` para definir la bandera `showpoint` y controlar los ceros a la derecha y la impresión del punto decimal en el caso de valores de punto flotante.

21.7.3 Justificación (ios::left, ios::right, ios::internal)

Las banderas `left` y `right` permiten que los campos sean justificados a la izquierda con caracteres de llenado a la derecha, o justificados a la derecha con caracteres de llenado a la izquierda, respectivamente. El carácter a utilizarse para el relleno se define mediante la función miembro `fill` o el manipulador de flujo parametrizado `setfill` (vea la sección 10.7.4). En la figura 21.22 se muestra el uso de los manipuladores `setw`, `setiosflags`, y `resetiosflags` así como las funciones miembro `setf` y `unsetf` para controlar la justificación a la derecha y a la izquierda de datos enteros dentro de un campo.

```
// fig21_21.cpp
// Controlling the printing of trailing
// zeros and decimal points when using
// floating point values with float values.

#include <iostream.h>
#include <iomanip.h>
#include <math.h>

main()
{
    cout << "cout prints 9.9900 as: " << 9.9900
        << "\ncout prints 9.9000 as: " << 9.9000
        << "\ncout prints 9.0000 as: " << 9.0000
        << "\n\nAfter setting the ios::showpoint flag\n";
    cout.setf(ios::showpoint);

    cout << "cout prints 9.9900 as: " << 9.9900
        << "\ncout prints 9.9000 as: " << 9.9000
        << "\ncout prints 9.0000 as: " << 9.0000 << endl;
    return 0;
}
```

```
cout prints 9.9900 as: 9.99
cout prints 9.9000 as: 9.9
cout prints 9.0000 as: 9

After setting the ios::showpoint flag
cout prints 9.9900 as: 9.990000
cout prints 9.9000 as: 9.900000
cout prints 9.0000 as: 9.000000
```

Fig. 21.21 Cómo controlar la impresión de ceros a la derecha y puntos decimales con valores de punto flotante.

```
// fig21_22.cpp
// Left-justification and right-justification.

#include <iostream.h>
#include <iomanip.h>

main()
{
    int x = 12345;

    cout << "Default is right justified:\n"
         << setw(10) << x
         << "\n\nUSING MEMBER FUNCTIONS\n"
         << "Use setf to set ios::left:\n" << setw(10);

    cout.setf(ios::left, ios::adjustfield);
    cout << x << "\nUse unsetf to restore default:\n";
    cout.unsetf(ios::left);
    cout << setw(10) << x
         << "\n\nUSING PARAMETERIZED STREAM MANIPULATORS"
         << "\nUse setiosflags to set ios::left:\n"
         << setw(10) << setiosflags(ios::left) << x
         << "\nUse resetiosflags to restore default:\n"
         << setw(10) << resetiosflags(ios::left)
         << x << endl;
    return 0;
}
```

```
Default is right justified:
      12345

USING MEMBER FUNCTIONS
Use setf to set ios::left:
      12345
Use unsetf to restore default:
      12345

USING PARAMETERIZED STREAM MANIPULATORS
Use setiosflags to set ios::left:
      12345
Use resetiosflags to restore default:
      12345
```

Fig. 21.22 Justificaciones a la izquierda y a la derecha.

La bandera **internal** indica que el signo de un número (o su base cuando la bandera **ios::showbase** está activa; vea la sección 21.7.5) deberá aparecer con justificación a la izquierda dentro de un campo, la magnitud del número deberá ser justificada a la derecha, y los espacios intermedios deberán ser rellenos con un carácter de llenado. Las banderas **left**, **right** e **internal** están contenidas en el miembro de datos estático **ios::adjustfield**. Al definir las banderas de justificación **left**, **right** o **internal**, el argumento **ios::adjustfield**

deberá ser proporcionado como segundo argumento de **setf**. Esto activará a **setf** a fin de asegurarse que solamente una de las tres banderas de justificación esté activa (son mutuamente excluyentes). En la figura 21.23 se muestra el uso de los manipuladores de flujo **setiosflags** y **setw** para definir el espaciado interno. Note el uso de la bandera **ios::showpos** para obligar a la impresión del signo más.

21.7.4 Relleno (Fill, Setfill)

La función miembro **fill** especifica el carácter de llenado a utilizarse en campos ajustados; si no se especifica valor, se utilizarán espacios para el relleno. La función **fill** devuelve el carácter de relleno anterior. El manipulador **setfill** también define el carácter de relleno. En la figura 21.24 se demuestra el uso de la función miembro **fill** y del manipulador **setfill** para controlar la definición y redefinición del carácter de llenado.

21.7.5 Base de flujo integral (ios::dec, ios::oct, ios::hex, ios::showbase)

El miembro estático **ios::basefield** (que se usa de una manera similar que **ios::adjustfield** con **setf**), incluye los bits **oct**, **hex** y **dec** para especificar que los enteros deben ser tratados como valores octales, hexadecimales y decimales, en forma respectiva. Por omisión, las inserciones de flujo se hacen a valores decimales, si ninguno de estos bits está activo; el valor por omisión para las extracciones de flujo es procesar los datos en la misma forma en que han sido introducidos —los enteros que se inicien con 0 se tratan como valores octales, los enteros que se inicien con 0x o 0X se tratarán como valores hexadecimales, y todos los demás enteros serán tratados como valores decimales. Una vez especificada una base particular para un flujo, todos los enteros de dicho flujo serán procesados con dicha base, hasta que se especifique una nueva base, o hasta el final del programa.

```
// fig21_23.cpp
// Printing an integer with internal spacing and
// forcing the plus sign.

#include <iostream.h>
#include <iomanip.h>

main()
{
    cout << setiosflags(ios::internal | ios::showpos)
         << setw(10) << 123 << endl;

    return 0;
}
```

```
+      123
```

Fig. 21.23 Cómo imprimir un entero con espaciado interno y obligando a la impresión del signo más.

```
// fig21_24.cpp
// Using the fill member function and the setfill
// manipulator to change the padding character for
// fields larger than the values being printed.

#include <iostream.h>
#include <iomanip.h>

main()
{
    int x = 10000;

    cout << x
        << " printed as int right and left justified\n"
        << "and as hex with internal justification.\n"
        << "Using the default pad character (space):\n";
    cout.setf(ios::showbase);
    cout << setw(10) << x << '\n';
    cout.setf(ios::left, ios::adjustfield);
    cout << setw(10) << x << '\n';
    cout.setf(ios::internal, ios::adjustfield);
    cout << setw(10) << hex << x << "\n\n";

    cout << "Using various padding characters:\n";
    cout.setf(ios::right, ios::adjustfield);
    cout.fill('*');
    cout << setw(10) << dec << x << '\n';
    cout.setf(ios::left, ios::adjustfield);
    cout << setw(10) << setfill('%') << x << '\n';
    cout.setf(ios::internal, ios::adjustfield);
    cout << setw(10) << setfill('^') << hex
        << x << endl;

    return 0;
}
```

```
10000 printed as int right and left justified
and as hex with internal justification.
Using the default pad character (space):
    10000
10000
0x  2710

Using various padding characters:
*****10000
10000%%%%%%%%
0x^^^^2710
```

Fig. 21.24 Cómo utilizar la función miembro `fill` y el manipulador `setfill` para modificar el carácter de relleno para campos mayores que los valores a imprimirse.

Active la bandera `showbase` para que se extraiga la base del valor completo. Los números decimales serán extraídos en forma normal, los números octales serán extraídos con un `0` a la izquierda, y los números hexadecimales serán extraídos ya sea con un `0x` a la izquierda o con un `0X` a la izquierda (la bandera `uppercase` determinará cuál de las opciones será seleccionada). En la figura 21.25 se demuestra el uso de la bandera `showbase` para obligar a que un entero se imprima en formatos decimal, octal y hexadecimales.

21.7.6 Números de punto flotante; notación científica (`ios::scientific`, `ios::fixed`)

La bandera `ios::scientific` y la bandera `ios::fixed` están incluidas en el miembro de dato estático `ios::floatfield` (que se usa en forma similar a `ios::adjustfield` e `ios::basefield` en `setf`). Se utilizan estas banderas para controlar el formato de salida de los números de punto flotante. La bandera `scientific` se activa para obligar a la salida de un número de punto flotante en formato científico. La bandera `fixed` obliga a un número de punto flotante a mostrar un número específico de dígitos (de acuerdo con lo especificado con la función miembro `precision`) a la derecha del punto decimal. Si estas banderas no están activadas, el valor del punto flotante determinará el formato de salida.

La llamada `cout.setf(0, ios::floatfield)` restaura el formato de sistema por omisión para la salida de número de punto flotante. En la figura 21.26 se demuestra la exhibición de los números de punto flotante, tanto en formatos fijos como científicos.

```
// fig21_25.cpp
// Using the ios::showbase flag

#include <iostream.h>
#include <iomanip.h>

main()
{
    int x = 100;

    cout << setiosflags(ios::showbase)
        << "Printing integers preceded by their base:\n"
        << x << '\n'
        << oct << x << '\n'
        << hex << x << endl;

    return 0;
}
```

```
Printing integers preceded by their base:
100
0144
0x64
```

Fig. 21.25 Cómo utilizar la bandera `ios::showbase`.

```
// fig21_26.cpp
// Displaying floating point values in system default,
// scientific, and fixed formats.

#include <iostream.h>

main()
{
    double x = .001234567, y = 1.946e9;

    cout << "Displayed in default format:\n"
          << x << '\t' << y << '\n';
    cout.setf(ios::scientific, ios::floatfield);
    cout << "Displayed in scientific format:\n"
          << x << '\t' << y << '\n';
    cout.unsetf(ios::scientific);
    cout << "Displayed in default format after unsetf:\n"
          << x << '\t' << y << '\n';
    cout.setf(ios::fixed, ios::floatfield);
    cout << "Displayed in fixed format:\n"
          << x << '\t' << y << endl;

    return 0;
}
```

```
Displayed in default format:
0.001235    1.946e+09
Displayed in scientific format:
1.234567e-03    1.946e+09
Displayed in default format after unsetf:
0.001235    1.946e+09
Displayed in fixed format:
0.001235    1946000000
```

Fig. 21.26 Cómo mostrar valores de punto flotante en formatos científicos, fijos y de sistema por omisión.

21.7.7 Control de mayúsculas/minúsculas (ios::uppercase)

La bandera `ios::uppercase` se activa para obligar la salida de una **X** o de una **E** en mayúsculas con enteros hexadecimales o con puntos con valores de punto flotante en notación científica, respectivamente (vea la figura 21.27). Cuando está activa, la bandera `ios::uppercase` hace que todas las letras en un valor hexadecimal aparezcan en mayúsculas.

21.7.8 Cómo activar y desactivar las banderas de formato (flags, setiosflags, resetiosflags)

La función miembro `flags`, sin argumento, sólo devuelve (como un valor `long`) los ajustes actuales de las banderas de formato. La función miembro `flags`, con un argumento `long`, define

```
// fig21_27.cpp
// Using the ios::uppercase flag
#include <iostream.h>
#include <iomanip.h>

main()
{
    cout << setiosflags(ios::uppercase)
          << "Printing uppercase letters in scientific\n"
          << "notation exponents and hexadecimal values:\n"
          << 4.345e10 << '\n'
          << hex << 123456789 << endl;

    return 0;
}
```

```
Printing uppercase letters in scientific
notation exponents and hexadecimal values:
4.345E+10
75BCD15
```

Fig. 21.27 Cómo utilizar la bandera `ios::uppercase`.

las banderas de formato según especificado en el argumento, y devuelve los ajustes de bandera anteriores. Cualquier otra bandera de formato que no se haya especificado en el argumento a `flags` será restablecida. El programa de la figura 21.28 demuestra el uso de la función miembro `flags` para definir un nuevo estado de formato, guardando el anterior, y a continuación restaurando los ajustes de formato original.

La función miembro `setf` define las banderas de formato incluidas en su argumento, y devuelve los ajustes previos de banderas como un valor `long`, como en

```
long previousFlagSettings =
    cout.setf(ios::showpoint | ios::showpos);
```

La función miembro `setf`, con dos argumentos `long`, como en

```
cout.setf(ios::left, ios::adjustfield);
```

primero desactiva los bits de `ios::adjustfield` y a continuación define la bandera `ios::left`. Esta versión de `setf` es utilizada con los campos de bits asociados con `ios::basefield` (representados por `ios::dec`, `ios::oct` e `ios::hex`), `ios::floatfield` (representado por `ios::scientific` e `ios::fixed`), e `ios::adjustfield` (representado por `ios::left`, `ios::right` e `ios::internal`).

La función miembro `unsetf` vuelve a activar las banderas designadas y devuelve el valor de las banderas existentes antes de que hayan sido reactivadas.

21.8 Estados de error de flujo

El estado de un flujo puede ser probado mediante bits en la clase `ios` —la clase base correspondiente a las clases `istream`, `ostream` e `iostream` que estamos utilizando para entradas/salidas.

```
// fig21_28.cpp
// Demonstrating the flags member function.
#include <iostream.h>

main()
{
    int i = 1000;
    double d = 0.0947628;

    cout << "The value of the flags variable is: "
        << cout.flags() << '\n'
        << "Print int and double in original format:\n"
        << i << '\t' << d << "\n\n";
    long originalFormat = cout.flags(ios::oct | ios::scientific);
    cout << "The value of the flags variable is: "
        << cout.flags() << '\n'
        << "Print int and double in a new format\n"
        << "specified using the flags member function:\n"
        << i << '\t' << d << "\n\n";
    cout.flags(originalFormat);
    cout << "The value of the flags variable is: "
        << cout.flags() << '\n'
        << "Print values in original format again:\n"
        << i << '\t' << d << endl;

    return 0;
}
```

```
The value of the flags variable is: 1
Print int and double in original format:
1000    0.094763

The value of the flags variable is: 4040
Print int and double in a new format
specified using the flags member function:
1750    9.47628e-02

The value of the flags variable is: 1
Print values in original format again:
1000    0.094763
```

Fig. 21.28 Demostración de la función miembro `flags`.

El `eofbit` queda automáticamente activado para un flujo de entrada cuando un fin de archivo es encontrado. Un programa puede utilizar la función miembro `eof` para determinar si en un flujo se ha encontrado un fin de archivo. La llamada

```
cin.eof()
```

devuelve verdadero si en `cin` se ha encontrado un fin de archivo, y falso de lo contrario.

El `failbit` queda activado para un flujo cuando en el mismo ocurre un error de formato, pero sin pérdida de caracteres. La función miembro `fail` determina si ha fallado la operación de un flujo; por lo regular es posible recuperarse de dichos errores.

El `badbit` se activa para un flujo cuando ocurre un error que resulte en pérdida de datos. La función miembro `bad` determina si ha fallado una operación de flujo. Estas fallas serias por lo regular no son recuperables.

El `goodbit` se activa en un flujo si para dicho flujo no se han activado ninguno de los bits `eofbit`, `failbit`, o `badbit`.

La función miembro `good` devuelve verdadero si las funciones `bad`, `fail` y `eof` devolvieran todas ellas falso. Las operaciones de entrada/salida deberían únicamente ser llevadas a cabo sobre flujos "buenos".

La función miembro `rdstate` devuelve el estado de error del flujo. Una llamada a `cout.rdstate`, por ejemplo, devolvería el estado del flujo que a continuación podría ser probado mediante un enunciado `switch` que examinase `ios::eofbit`, `ios::badbit`, `ios::failbit` e `ios::goodbit`. La manera preferida de probar el estado de un flujo es utilizando las funciones miembro `eof`, `bad`, `fail` y `good` —el uso de estas funciones no requiere que el programador esté familiarizado con bits de estado particulares.

La función miembro `clear` se utiliza por lo general, para restaurar el estado de un flujo a "bueno", de tal forma que puedan continuar las entradas/salidas sobre dicho flujo. El argumento por omisión para `clear` es `ios::goodbit`, por lo que el enunciado

```
cin.clear();
```

elimina o limpia `cin` y activa `goodbit` para el flujo. El enunciado

```
cin.clear(ios::failbit)
```

es el que verdaderamente activa `failbit`. El usuario quizás deseara hacer esto al ejecutar una entrada en `cin` con un tipo definido por usuario y al encontrar un problema. En este contexto el nombre `clear` parece inapropiado, pero es correcto.

El programa de la figura 21.29 ilustra el uso de las funciones miembro `rdstate`, `eof`, `fail`, `bad`, `good`, y `clear`.

La función miembro `operator!` devuelve verdadero ya sea que `badbit` esté activo, `failbit` o ambos. La función miembro `operator void*` devuelve falso si `badbit`, `failbit` o ambos están activos. Estas funciones resultan útiles en el procesamiento de archivos, cuando se hacen pruebas de condiciones verdaderas/falsas dentro de la condición de una estructura de selección o de repetición.

21.9 Entradas/salidas de tipos definidos por usuario

C++ es capaz de introducir y extraer los tipos de datos estándar utilizando los operadores de extractor de flujo `>>` y de inserción de flujo `<<`. Estos operadores son homónimos, a fin de poder procesar cada tipo de datos estándar, incluyendo cadenas y direcciones en memoria. El programador puede hacer la homonimia de los operadores de inserción y de extracción de flujo, a fin de llevar a cabo entradas y salidas de tipos definidos por usuario. En la figura 21.30 se demuestra la homonimia de operadores de extracción y de inserción de flujo, a fin de manejar datos de una clase de número telefónicos definidos por usuario llamada `PhoneNumber`. Note que este programa supone que los números telefónicos están introducidos correctamente. Dejamos como ejercicio el incorporar la correspondiente verificación de errores.

```
// fig21_29.cpp
// Testing error states.

#include <iostream.h>

main()
{
    int x;
    cout << "Before a bad input operation:\n"
    << "cin.rdstate(): " << cin.rdstate()
    << "\n    cin.eof(): " << cin.eof()
    << "\n    cin.fail(): " << cin.fail()
    << "\n    cin.bad(): " << cin.bad()
    << "\n    cin.good(): " << cin.good()
    << "\n\nExpects an integer, but enter a character: ";
    cin >> x;

    cout << "\nAfter a bad input operation:\n"
    << "cin.rdstate(): " << cin.rdstate()
    << "\n    cin.eof(): " << cin.eof()
    << "\n    cin.fail(): " << cin.fail()
    << "\n    cin.bad(): " << cin.bad()
    << "\n    cin.good(): " << cin.good() << "\n\n";

    cin.clear();

    cout << "After cin.clear()\n"
    << "cin.fail(): " << cin.fail() << endl;

    return 0;
}
```

```
Before a bad input operation:
cin.rdstate(): 0
cin.eof(): 0
cin.fail(): 0
cin.bad(): 0
cin.good(): 1

Expects an integer, but enter a character: A

After a bad input operation:
cin.edstate(): 2
cin.eof(): 0
cin.fail(): 2
cin.bad(): 0
cin.good(): 0

After cin.clear()
cin.fail(): 0
```

Fig. 21.29 Cómo probar estados de error.

El operador de extracción de flujo toma como argumentos una referencia **istream** y una referencia a un tipo definido por usuario (en este caso **PhoneNumber**), y devuelve una referencia **istream**. En la figura 21.30, el operador de extracción de flujo homónimo es utilizado para introducir los números telefónicos de la forma

```
(800) 555-1212)
```

en objetos del tipo **PhoneNumber**. La función de operador lee las tres partes de un número telefónico en los miembros **areaCode**, **exchange** y **line** del objeto referenciado **PhoneNumber** (**num** en la función de operador y **phone** en **main**). Los caracteres de paréntesis, de espacio y de guiones serán descartados mediante el llamado a la función miembro **ignore istream**. La función de operador devuelve la referencia **input istream**. Al devolver la referencia de flujo, las operaciones de entrada en objetos **PhoneNumber** pueden ser concatenadas con operaciones de entrada sobre otros objetos **PhoneNumber** o con otros tipos de datos. Por ejemplo, dos objetos **PhoneNumber** podrían ser introducidos como sigue:

```
cin >> phone1 >> phone2;
```

El operador de inserción de flujo toma como argumentos una referencia **ostream** y una referencia a un tipo definido por usuario (en este caso **PhoneNumber**), y devuelve una referencia **ostream**. En la figura 21.30, el operador de inserción de flujo homónimo exhibe objetos del tipo **PhoneNumber**, de la misma forma en que fueron introducidos. La función **operator** exhibe las partes del número telefónico como cadenas, porque están almacenados en formato de cadena (recuerde que la función miembro **getline** de **istream** almacena un carácter nulo después de que termina su entrada).

Las funciones homónimas **operator** se declaran en **class PhoneNumber** como funciones **friend**. Los operadores de entrada y de salida homónimos deben de ser declarados como **friend**, para que puedan tener acceso a miembros de clase no públicos. Los amigos de una clase pueden tener acceso a los miembros de clase privados.

Observación de ingeniería de software 21.3

Se pueden añadir a C++ nuevas capacidades de entrada/salida para tipos definidos por usuario, sin modificar las declaraciones o los miembros de datos privados, ya sea para la clase **ostream** o para la clase **istream**. Esto fomenta la extensibilidad del lenguaje de programación C++ —lo que es uno de los aspectos más atractivos de C++.

21.10 Cómo ligar un flujo de salida con un flujo de entrada

Las aplicaciones interactivas generalmente involucran un **istream** como entrada y un **ostream** como salida. Cuando en la pantalla aparece un mensaje solicitando algo, el usuario responde introduciendo los datos apropiados. Obviamente, la solicitud debe aparecer antes de que se inicie la operación de entrada. Con almacenamiento temporal de salida, las salidas sólo aparecerán cuando el búfer esté lleno, cuando las salidas se vacíen en forma explícita por requerimientos del programa, o bien en forma automática al final del programa. C++ tiene la función miembro **tie** para sincronizar, es decir, "para ligar" la operación de un **istream** con un **ostream** para asegurar que las salidas aparezcan antes de sus entradas subsecuentes. Una llamada como

```
// fig21_30.cpp
// User-defined insertion and extraction operators

#include <iostream.h>

class PhoneNumber {
    friend ostream& operator<<(ostream&, PhoneNumber&);
    friend istream& operator>>(istream&, PhoneNumber&);
private:
    char areaCode[4];
    char exchange[4];
    char line[5];
};

ostream& operator<<(ostream& output, PhoneNumber& num)
{
    output << "(" << num.areaCode << " "
        << num.exchange << "-" << num.line;

    return output;
}

istream& operator>>(istream& input, PhoneNumber& num)
{
    input.ignore();           // skip (
    input.getline(num.areaCode, 4);
    input.ignore(2);         // skip ) and space
    input.getline(num.exchange, 4);
    input.ignore();          // skip -
    input.getline(num.line, 5);

    return input;
}

main()
{
    PhoneNumber phone;

    cout << "Enter a phone number in the "
        << "form (123) 456-7890:\n";
    cin >> phone;
    cout << "The phone number entered was:\n"
        << phone << endl;

    return 0;
}
```

```
Enter a phone number in the form (123) 456-7890:
(800) 555-1212
The phone number entered was:
(800) 555-1212
```

Fig. 21.30 Operadores de inserción y de extracción de flujo definidos por usuario.

```
cin.tie(cout);
```

Liga a `cout` (un `ostream`) con `cin` (un `istream`). De hecho, esta llamada en particular resulta redundante, porque C++ lleva a cabo esta operación en forma automática para crear el entorno estándar de entrada/salida. El usuario podría, sin embargo, ligar en forma explícita otros pares `istream/ostream`. A fin de desligar un flujo de entrada, `inputStream`, de un flujo de salida, utilice la llamada

```
inputStream.tie(0);
```

Resumen

- Las operaciones de entrada/salida se ejecutan de una forma que depende o es sensible al tipo de los datos.
- Las entradas y salidas en C++ ocurren en flujos de bytes. Un flujo es sólo una secuencia de bytes.
- Los mecanismos de entrada/salida del sistema mueven bytes de dispositivos a la memoria y viceversa de una forma eficiente y confiable.
- C++ proporciona capacidades de entrada/salida de “bajo nivel” y de “alto nivel”. Las capacidades de entrada/salida de bajo nivel especifica que cierto número de bytes deberán ser transferidos del dispositivo a la memoria o de la memoria al dispositivo. Las entradas/salidas de alto nivel se ejecutan con bytes agrupadas en unidades significativas como son enteros, puntos flotantes, caracteres, cadenas y tipos definidos por usuario.
- C++ proporciona operaciones de entradas/salidas tanto con como sin formato. Las entradas y salidas sin formato son rápidas, pero procesan datos en bruto difíciles de utilizar por las personas. Las entradas/salidas con formato procesan los datos en unidades significativas, pero requieren de tiempo de proceso adicional que puede tener un impacto negativo en transferencias de datos a altos volúmenes.
- La mayor parte de los programas de C++ incluyen un archivo de cabecera `iostream.h` que contiene la información básica requerida para todas las operaciones de entradas/salidas de flujo.
- El encabezado `io manip.h` contiene información para entradas/salidas con formato mediante manipuladores de flujo parametrizados.
- El encabezado `fstream.h` contiene información para operaciones de procesamiento de archivos.
- El encabezado `strstream.h` contiene información para dar formato en memoria.
- El encabezado `stdiostream.h` contiene información para aquellos programas que mezclan los estilos de entradas/salidas de C con las de C++.
- La clase `istream` acepta operaciones de entrada de flujo.
- La clase `ostream` acepta operaciones de salida de flujo.
- La clase `iostream` acepta tanto operaciones de entrada como operaciones de salida de flujo.
- Las clases `istream` y `ostream` son ambas derivadas mediante herencia simple a partir de la clase base `ios`.
- La clase `iostream` es derivada mediante herencia múltiple, tanto de la clase `istream` como de la clase `ostream`.

- Se hace la homonimia del operador de desplazamiento a la izquierda (<<), para designar salida de flujo y se le conoce como el operador de inserción de flujo.
- Se hace la homonimia del operador de desplazamiento a la derecha (>>) para designar entrada de flujo y se le conoce como el operador de extracción de flujo.
- El objeto **cin** de la clase **istream** está ligado al dispositivo de entrada estándar, que por lo general, es el teclado.
- El objeto **cout** de la clase **ostream** está ligado al dispositivo de salida estándar, que por lo regular es la pantalla de exhibición.
- El objeto **cerr** de la clase **ostream** está ligado con el dispositivo de error estándar. Las salidas a **cerr** no pasan por búfer; cada inserción a **cerr** aparece de inmediato.
- El manipulador de flujo **endl** emite un carácter de nueva línea y vacía el búfer de salida.
- El compilador de C++ determina automáticamente los tipos de datos para la entrada y la salida.
- Por omisión las direcciones se exhiben en formato hexadecimal.
- Para imprimir la dirección de una variable del tipo **char***, convierta explícitamente (cast) el apuntador a **void***.
- La función miembro **put** extrae un carácter. Las llamadas a **put** pueden ser concatenadas.
- La salida de flujo se lleva a cabo mediante el operador de extracción de flujo >>. Este operador en forma automática pasa por alto caracteres de espacio en blanco existentes en el flujo de entrada.
- El operador >> devuelve falso cuando en un flujo encuentra la señal de fin de archivo.
- La extracción de flujo hace que se active **failbit** cuando se trate de una entrada incorrecta, y que se active **badbit** si la operación falla.
- Una serie de valores pueden ser introducidos mediante la operación de extracción de flujo en un encabezado de ciclo **while**. La extracción devolverá falso (cero) cuando se encuentre fin de archivo.
- La función miembro **get**, sin argumentos, introduce un carácter y devuelve el carácter; si encuentra en el flujo la señal de fin de archivo, se devolverá **EOF**.
- La función miembro **get**, con un argumento del tipo **char**, introduce un carácter. Se devolverá falso al encontrar el fin de archivo; de lo contrario será devuelto el objeto **istream** para el cual se está invocando la función miembro **get**.
- La función miembro **get**, con tres argumentos, un arreglo de caracteres, un límite de tamaño y un delimitador (con el valor de nueva línea por omisión) —lee caracteres del flujo de entrada hasta un máximo de límite - 1 caracteres y termina, o termina cuando se lea el delimitador. La cadena de entrada se terminará incluyendo un carácter nulo. El delimitador no se coloca en el arreglo de caracteres, sino que se conserva dentro del flujo de entrada.
- La función miembro **getline** opera en forma similar a la función miembro **get** con tres argumentos. La función **getline** elimina el delimitador del flujo de entrada.
- La función miembro **ignore** pasa por alto el número específico de caracteres (su valor por omisión es un carácter) dentro del flujo de entrada; termina si encuentra el delimitador especificado (el delimitador por omisión es **EOF**).
- La función miembro **putback** coloca el carácter previamente obtenido por un **get** de un flujo de regreso en dicho flujo.

- La función miembro **peek** devuelve el siguiente carácter de un flujo de entrada, pero sin eliminar dicho carácter de un flujo.
- C++ ofrece entrada/salida de tipo seguro. Si se procesan datos inesperados por los operadores << y >>, se definirán varias banderas de error, que el usuario podrá probar para determinar si una operación de entrada/salida ha tenido éxito o ha fallado.
- Las entradas/salidas sin formato se llevan a cabo con las funciones miembro **read** y **write**. Estas introducen o extraen un cierto número de bytes, hacia o de la memoria, empezando en una dirección designada de memoria. Se introducen o se extraen como bytes en bruto sin formato.
- La función miembro **gcount** devuelve el número de caracteres introducidos mediante la operación anterior **read** sobre dicho flujo.
- La función miembro **read** introduce un número especificado de caracteres en un arreglo de caracteres. Si se han leído menos que el número especificado de caracteres, se activa **failbit**.
- Para modificar la base numérica en la cual se extraen enteros, utilice el manipulador **hex** para definir la base a hexadecimal (de base 16) o bien **oct** para definir la base a octal (de base 8). Utilice el manipulador **dec** para restaurar la base a decimal. La base se conservará sin modificación en tanto no sea cambiada en forma explícita.
- El manipulador de flujo parametrizado **setbase** también define la base de la salida de enteros. **setbase** toma un argumento entero de 10, 8, ó 16 para definir la base.
- La precisión de punto flotante puede ser controlada utilizando o el manipulador de flujo **setprecision** o la función miembro **precision**. Ambos definen la precisión para todas las operaciones de salida subsecuentes hasta la siguiente llamada de ajuste de precisión. La función miembro **precision**, sin argumentos, devuelve el valor de precisión actual. Una precisión de 0 define el valor de precisión por omisión (6).
- Los manipuladores parametrizados requieren la inclusión del archivo de cabecera **iomanip.h**.
- La función miembro **width** define el ancho de campo y devuelve el ancho anterior. Los valores que sean menores que el campo serán llenados con caracteres de relleno. El ajuste de ancho de campo se aplica sólo para la siguiente inserción o extracción; después el ancho de campo se define en forma implícita a 0 (los valores subsecuentes serán extraídos del tamaño que necesiten tener). Los valores mayores que el tamaño del campo son impresos por completo. La función **width** sin argumentos, devuelve el ajuste actual de ancho. El manipulador **setw** también define el ancho.
- Para las entradas, el manipulador de flujo **setw** establece un tamaño máximo de cadena; si se introduce una cadena más grande, la línea más grande se divide en piezas no mayores que el tamaño designado.
- Los usuarios pueden crear sus propios manipuladores de flujo.
- Las funciones miembro **setf**, **unsetf**, y **flags** controlan los ajustes de bandera.
- La bandera **skipws** indica que en un flujo de entrada el operador >> deberá pasar por alto los espacios en blanco. El manipulador de flujo **ws** también pasa por alto los espacios en blanco a la izquierda en un flujo de entrada.
- Las banderas de formato se definen como una enumeración en la clase **ios**.
- Las banderas de formato están controladas por las funciones miembro **flags**, y **setf**, pero muchos programadores de C++ prefieren utilizar manipuladores de flujo. La operación OR a